

Project Report

Student Issue Tracker (Complaint Management System)

Submitted by

BSCSE-22-05

Affifa Fatima

Session 2022-2026

Supervised by

Dr. Aamir Raza



DEPARTMENT OF COMPUTER SCIENCE
BAHAUDDIN ZAKARIYA UNIVERSITY MULTAN

FINAL APPROVAL

This is to certify that we have read this report titled **Student Issue Tracker (Complaint Management System)** submitted by **Affifa Fatima (BSCSE-22-05)** and it is our judgment that this report is of sufficient standard to warrant its acceptance by Bahauddin Zakariya University, Multan for the degree of BS (Computer Science).

Committee:

1. External Examiner

<<Examiner Name>>

<<Designation>>

<<Organization>>

2. Supervisor

Dr. Amir Raza

Assistant Professor,

Department of Computer Science

Bahauddin Zakariya University, Multan

3. Chairperson

Dr. Humaira Afzal

Professor

Department of Computer Science

Bahauddin Zakariya University, Multan.

DEDICATION

To my beloved father, whose unwavering support, sacrifices, and constant encouragement have been the foundation of my success. His guidance, strength, and belief in me have always inspired me to strive for excellence.

This work is wholeheartedly dedicated to him as a small token of my deepest gratitude and respect.

ACKNOWLEDGMENT

All praise and gratitude belong to Allah Almighty, the Most Gracious and Most Merciful, whose countless blessings and guidance enabled me to successfully complete this Final Year Project. I would like to express my sincere and heartfelt gratitude to my respected supervisor, **Dr. Aamir Raza**, for his continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His support and mentorship played a vital role in shaping this work and enhancing my overall learning experience.

I am also thankful to the faculty members of the Department of Computer Science, **Bahauddin Zakariya University**, for their academic support and for providing a strong foundation throughout my undergraduate studies. I am deeply grateful to my family for their unconditional love, prayers, and patience. Their constant encouragement and belief in me have always been my greatest source of motivation and strength.

PROJECT BRIEF

PROJECT NAME	Student Issue Tracker (Complaint Management System)
ORGANIZATION NAME	Department of Computer Science, Bahauddin Zakariya University, Multan
UNDERTAKEN BY	Affifa Fatima (BSCSE-22-05)
SUPERVISED BY	Dr. Rana Aamir Raza
STARTING DATE	October 01, 2025
COMPLETION DATE	April 27, 2026
COMPUTER USED	Intel Core i5-6200U, 8GB RAM , 23GB ,SSD
OPERATING SYSTEM	Windows 11
SOURCE LANGUAGE(S)	JavaScript (Next.js/React), CSS, SQL, Framer Motion
DBMS USED	Supabase PostgreSQL (Development/Production)
TOOLS/PACKAGES	Next.js API Routes, Supabase Auth/Storage, Recharts, bcryptjs, Botpress, VS Code, Postman, Git/GitHub

ABSTRACT

The AI-Enhanced Student Issue Tracker is a web-based complaint management platform designed to streamline issue handling in university departments.

? It supports multiple user roles including Student, Faculty Member, HOD, DSA, and Supervisor, ensuring a structured workflow from submission to closure.

? The system includes AI-assisted category suggestion, severity/priority estimation, duplicate-aware submission support, attachment handling, notifications, analytics, activity logs, weekly reporting, and chatbot assistance.

? Complaints are routed by category and department: Academic, Facilities, Behavior-related, and Other issues normally route to HOD, while Administrative issues route to DSA. HOD/DSA can review, reject, or assign work to faculty; faculty resolve cases; DSA/Supervisor close finalized complaints; students can rate the resolution.

Developed using Next.js, React, Supabase Auth, Supabase PostgreSQL, and server-side Next.js API routes, the system improves transparency, accountability, and resolution efficiency.

Contents

<i>Committee:</i>	i
ACKNOWLEDGMENT.....	iii
Student Issue Tracker (Complaint Management System)	iv
ABSTRACT.....	v
1.1 Project Introduction	1
1.1.1 Main Theme	1
1.1.2 Scope of the Project	1
1.1.3 Objectives of the Project.....	2
1.1.4 Problem Statement	2
1.1.5 Significance of the Project	2
1.1.6 Methodology	3
1.1.7 Limitations of the Project.....	3
1.2 Introduction to Organization.....	3
1.2.1 Organizational Setup and Structure	3
1.2.2 Main Aim and Work Environment	4
1.3 Conclusion and Future Prospectus.....	4
1.4 Thesis Organization	4
1.5 Background and Motivation	4
1.6 System Architecture Overview	5
2.1 Feasibility Study	9
2.1.1 Technical Feasibility	9
2.1.2 Operational Feasibility	9
2.1.3 Economic Feasibility	9
2.1.4 Schedule Feasibility	9

2.1.5 Legal and Security Feasibility	9
2.2 Existing System: Data Gathering.....	10
2.2.1 Questionnaires.....	10
2.2.2 Sampling and Observations	10
2.2.3 Interviews.....	10
2.3 Existing System: Data Analysis.....	10
2.3.1 Data Flow Diagrams (DFDs)	10
2.3.2 Requirements Engineering.....	12
2.3.3 Deliverables	13
2.4 Proposed System Overview	13
2.5 Assumptions and Constraints.....	13
3.1 Introduction to System Design.....	15
3.2 Proposed System and its Features.....	15
3.3 System Design using UML	15
3.3.1 Use Case Diagrams	15
3.3.2 Sequence Diagrams.....	16
3.4 Database Design.....	20
3.4.1 Entity Relationship Diagram (ERD).....	20
3.4.2 Relational Model from ERDs	20
3.4.3 Normalizing the Relational Model to 3NF	21
4.1 Introduction to System Development	24
4.2 Tool and Language Selection.....	24
4.3 Hardware Requirements.....	24
4.4 Software Development and Implementation	25
4.4.1 Web Portal (Frontend — React)	25

4.4.2 Backend (Server-Side — Django REST)	25
4.4.3 Database Technologies	25
4.5 Algorithms of Important Modules	25
4.5.1 Role-Based Access Control (RBAC) Workflow	25
4.5.2 AI Category Prediction Workflow.....	26
4.5.3 Severity Scoring Engine	26
4.5.4 Duplicate Detection (Semantic Similarity).....	26
4.5.5 HOD Review and Admin Assignment Workflow	26
4.5.6 Escalation Module	26
4.5.7 Weekly Reporting Module.....	27
4.5.8 AI Module Integration Architecture	27
4.5.9 Email Notification Architecture.....	28
5.1 Input Forms	30
5.1.1 Registration Screen	30
5.1.2 Login Screen (Web).....	30
5.1.3 Complaint Submission Form (Student)	31
5.1.4 HOD Review Interface	32
5.1.5 Teacher Assignment Modal (Admin)	33
5.1.6 Teacher Solution / Comment Form.....	34
5.2 Reports and Dashboards	35
5.2.1 Role-Based Dashboards	35
5.2.2 Analytic Reports	42
5.2.3 Chatbot and Weekly Reports	46
5.2.4 Activity Feed and History	47
5.2.5 AI Processing Results Report	48

6.1 Project Summary.....	51
6.1.1 Summary of Work Completed.....	51
6.2 Project Outcomes Summary	51
6.3 Key Achievements.....	51
6.4 Limitations	51
6.5 Future Work and Recommendations	52
6.6 Personal Reflection	52
References.....	54
Appendix A: Technology Stack Summary	55
Appendix B: Key Implemented Backend Files	56

Chapter 1

INTRODUCTION

1.1 Project Introduction

Universities receive a wide range of student complaints related to academics, administration, hostel services, transport, faculty behavior, and IT support. In many institutions these complaints are still managed through informal channels, creating delays, missing records, and weak accountability. This Complaint Management System addresses this gap by providing a full-stack web application with separate dashboards for Students, HOD, Admin, and Teacher users.

1.1.1 Main Theme

The central theme is the automation and digitization of department-level complaint management. The system governs the following core processes:

- Complaint submission with AI-based category prediction and severity estimation.
- Semantic duplicate detection to prevent redundant complaint entries.
- HOD-level first-gate review with accept/reject workflow.
- Admin-level teacher assignment and complaint finalization.
- Teacher-level complaint resolution with solution comments.
- Automated email notifications across all key workflow events.
- Role-based analytics dashboards for all four user types.
- Weekly PDF reporting and scheduled escalation logic.
- Botpress chatbot integration for student guidance.

1.1.2 Scope of the Project

The scope encompasses student, HOD, admin, and teacher-facing functionality through a single web application. Student module includes registration, login, complaint submission with AI results, tracking, and chatbot. HOD module includes pending review with accept/reject. Admin module includes teacher assignment, finalization, activity logs, analytics, and weekly reports. Teacher module includes assigned complaint queue, status updates, and solution comments.

1.1.3 Objectives of the Project

1. To design and implement a web-based AI-enhanced department complaint management system using Next.js, React, and Supabase.
2. To provide secure role-based registration and login for Student, Faculty Member, HOD, DSA, and Supervisor.
3. To suggest complaint category and priority during submission and support duplicate-aware complaint handling.
4. To implement a structured workflow from student submission to HOD/DSA review, faculty assignment, resolution, closure, and student feedback.
5. To provide role-based dashboards, notifications, analytics, activity tracking, and weekly reports for transparent decision-making.
6. To provide chatbot-assisted guidance for student users throughout the complaint process.
8. To provide chatbot-assisted guidance for student users throughout the complaint process.

1.1.4 Problem Statement

Many academic departments lack a centralized and intelligent complaint management platform. Existing complaint handling suffers from informal submission channels, difficult status tracking, duplicate complaints creating extra workload, absence of automatic prioritization, unclear role responsibilities, limited student visibility into progress, and no analytics for administrators. As a result, complaints are frequently delayed, misrouted, or ignored, causing dissatisfaction and departmental inefficiency.

1.1.5 Significance of the Project

- Students: Structured complaint submission, AI categorization, duplicate prevention, transparent tracking, and chatbot guidance.
- HOD: First-level review with clear accept/reject controls and escalation alerts for overdue cases.
- Admin: Centralized assignment management, activity logging, weekly reports, and performance analytics.
- Teachers: Focused complaint workload with clear resolution responsibilities and solution documentation.
- Academic Value: Demonstrates full-stack development, ML integration, semantic NLP, RBAC, and workflow automation.

1.1.6 Methodology

9. Phase 1 — Requirements Gathering: Comprehensive understanding of academic department complaint workflows, multi-role access patterns, and AI integration requirements.
10. Phase 2 — System Analysis and Design: UML modeling including use case and sequence diagrams; ERD design and normalization to 3NF.
11. Phase 3 — Technology Selection: React, Django REST Framework, SQLite, scikit-learn, sentence-transformers, JWT, ReportLab, and Botpress.
12. Phase 4 — Implementation: All modules implemented including authentication, AI pipeline, HOD review, admin assignment, teacher resolution, analytics, reporting, and chatbot.
13. Phase 5 — Testing and Validation: Functional testing across all user roles and workflow paths.
14. Phase 6 — Documentation: System documentation, user guide, and final thesis report.

1.1.7 Limitations of the Project

- No mobile application support in the current version.
- Single-department deployment; multi-department extension not yet implemented.
- No document or image attachment support for complaints.
- Real-time WebSocket notifications not implemented; email-based notifications used.
- SQLite is suitable for development but not optimized for high-scale production.

1.2 Introduction to Organization

This project has been developed and submitted as part of the Final Year Project requirements for the Bachelor of Science in Computer Science program at Bahauddin Zakariya University (BZU), Multan. The work applies software engineering practices to the intelligent and automated management of student complaints within an academic department.

1.2.1 Organizational Setup and Structure

Bahauddin Zakariya University (BZU), Multan is a leading public sector university. The Department of Computer Science provides structured coursework and project supervision. Final Year Projects are supervised by experienced faculty and evaluated through formal academic processes including periodic reviews and a final defense presentation.

1.2.2 Main Aim and Work Environment

The primary aim of the Department of Computer Science is to produce graduates equipped with strong theoretical foundations and practical development competencies. This system was designed and implemented within this academic environment using iterative development practices.

1.3 Conclusion and Future Prospectus

This Complaint Management System successfully addresses core complaint handling challenges through a centralized, role-based, AI-supported web platform. Looking ahead, future enhancements include mobile application development, multi-department deployment, document attachments, real-time notifications, PostgreSQL migration, and improved NLP-based severity scoring.

1.4 Thesis Organization

- Chapter 2 — System Analysis: Feasibility study, data gathering, DFD diagrams, and requirements engineering.
- Chapter 3 — System Design: UML diagrams, ERD, relational model, and normalization (1NF, 2NF, 3NF).
- Chapter 4 — System Development and Implementation: Technology stack, algorithms, and module implementation.
- Chapter 5 — User's Guide: Role-based dashboards, forms, and interface descriptions.
- Chapter 6 — Conclusion: Summary, achievements, limitations, and future work.

1.5 Background and Motivation

The development of this system is motivated by the growing recognition within academic institutions that student-facing administrative processes must be modernized and digitized to meet contemporary expectations for transparency, efficiency, and data-driven decision-making. Research literature consistently identifies complaint management as one of the most impactful student experience factors within higher education institutions, with poor complaint handling correlating strongly with decreased academic engagement and student satisfaction scores.

Traditional approaches to complaint management in university departments — primarily paper-based forms, verbal reporting channels, and unstructured email threads — share a set of fundamental operational limitations that technology can directly address. These limitations include: lack of centralized record-keeping, preventing consistent tracking and follow-up; absence of formal status tracking, leaving students without visibility into the progress of their submissions; inconsistent role assignment, causing complaints to be routed to the wrong person or lost between multiple parties; no systematic prioritization mechanism, treating urgent complaints identically to routine administrative requests; and no aggregate analytics capability, preventing administrators from identifying recurring problems or monitoring staff resolution performance over time.

The emergence of practical machine learning toolkits (scikit-learn, transformers), accessible natural language processing libraries (sentence-transformers), and mature REST API development frameworks (Django REST Framework) now makes it feasible for a small development team to implement an AI-enhanced complaint management system without requiring specialized AI engineering expertise or enterprise-scale infrastructure. This accessibility makes the integration of AI assistance into departmental complaint management a realistic and valuable enhancement — not merely a theoretical concept.

This project was specifically designed to be deployable within a real academic department environment, using technology choices calibrated for reliability, maintainability, and graceful degradation in the event of AI component unavailability — reflecting the practical constraints of academic IT environments where infrastructure may be limited and dedicated DevOps support is rare.

1.6 System Architecture Overview

The system is structured as a three-tier client-server architecture with an AI service layer integrated within the server tier. The three tiers are as follows:

Presentation Tier (Next.js/React Frontend): The frontend is implemented with Next.js App Router and React components. It provides the landing page, registration/login screens, role-aware dashboard, complaint form, complaint cards, analytics panels, notifications, Botpress student assistant, and responsive user interface components. Client-side helpers call protected API routes and Supabase session handling keeps each user inside the dashboard scope allowed for their role.

Application Tier (Next.js API Routes): Server-side API routes provide the business workflow for authentication profile lookup, complaint creation, review, assignment, status updates, finalization, ratings, comments, attachments, analytics, activity logs, notifications, departments, staff management, and weekly reports. All protected routes validate the Supabase session token and then enforce role permissions before running the requested action.

Data Tier (Supabase PostgreSQL): Supabase stores profiles, departments, complaints, complaint attachments, comments, notifications, category routes, and activity logs. PostgreSQL enums define roles, statuses, categories, priorities, and routed roles. Row Level Security policies protect user-owned records, while trusted server-side API routes use the service role for cross-role workflows such as staff assignment, analytics, and supervisor operations.

Workflow Layer: Category routing maps Academic, Behavior-related, Facilities, and Other complaints to HOD by default, and Administrative complaints to DSA. Students submit complaints with optional attachments. HOD/DSA users review and accept/reject submitted complaints, then assign accepted work to faculty members. Faculty members update assigned complaints and mark them resolved. DSA or Supervisor closes finalized complaints, after which students may provide rating and feedback.

Support Services: The system includes email/notification helpers, analytics summaries, activity tracking, weekly reporting, password validation, Botpress chatbot support, and reusable UI components for status badges, role badges, timelines, AI panels, and complaint cards.

blocks that catch loading or inference exceptions and return appropriate fallback values, ensuring that AI unavailability never prevents complaint submission.

Chapter 2

SYSTEM ANALYSIS

2.1 Feasibility Study

A feasibility study was conducted to verify that the system could be designed and deployed within available time, skills, and resource constraints. The study examined five feasibility dimensions.

2.1.1 Technical Feasibility

- React for frontend interface and role-based dashboard development.
- Django REST Framework for secure backend API development.
- scikit-learn and joblib for ML category prediction.
- sentence-transformers for semantic duplicate detection.
- JWT via djangorestframework-simplejwt for authentication.
- ReportLab for PDF weekly report generation.

2.1.2 Operational Feasibility

The system aligns with actual departmental complaint workflows. The four defined roles — Student, HOD, Admin, and Teacher — reflect common academic structures. Familiar UI patterns such as forms, dashboards, and notification cards reduce the learning curve significantly.

2.1.3 Economic Feasibility

The software stack is largely open-source, reducing development cost significantly. The system replaces manual complaint handling overhead and improves administrative efficiency, supporting a positive cost-benefit case for departmental deployment.

2.1.4 Schedule Feasibility

The modular architecture allowed separate implementation of authentication, complaint workflow, AI services, analytics, and chatbot within manageable development phases. Core modules were prioritized first with AI and analytics added incrementally.

2.1.5 Legal and Security Feasibility

Feasibility is supported through JWT-based authentication, role-based permission classes, protected frontend routes, optional anonymous submission, and activity logging for auditability.

2.2 Existing System: Data Gathering

2.2.1 Questionnaires

- Current complaint submission methods and their limitations.
- Difficulty tracking complaint status and expected resolution times.
- Problems caused by duplicate or redundant complaint submissions.
- Desired features: automatic prioritization, anonymous submission, and email updates.

2.2.2 Sampling and Observations

- Manual complaint submission delays and tracking difficulties.
- Inconsistent role responsibilities for complaint routing.
- Lack of visibility into complaint progress for students.
- Absence of analytics or reporting on complaint volumes.

2.2.3 Interviews

Interviews captured informal processes and edge cases including complaints requiring escalation, complaints rejected due to vague descriptions, and cases where duplicate submissions caused confusion. These shaped design decisions for severity scoring, duplicate detection thresholds, and the escalation scheduler.

2.3 Existing System: Data Analysis

2.3.1 Data Flow Diagrams (DFDs)

Data Flow Diagrams were used to visualize information flow, identify bottlenecks, and map automation opportunities within the existing manual complaint process.

DFD Level 0 (Context Diagram) shows the system as a single process bubble interacting with four external entities: Student, HOD, Admin, and Teacher. Each entity exchanges specific data flows with the central system representing the complete complaint lifecycle.

Figure 2.1 — DFD Level 0: Context Diagram
AI-Enhanced Department Complaint Management System



External entities interact with the central system through role-specific data flows covering complaint submission, review, assignment, resolution, analytics, and reporting.

Figure 2.1: DFD Level 0 — Context Diagram, Complaint Management System

DFD Level 1 (Process Decomposition) decomposes the system into ten major processes: Authentication & Access Control (P1), Complaint Submission (P2), AI Services — Category/Severity/Duplicate (P3), HOD Review Workflow (P4), Admin Management & Assignment (P5), Teacher Resolution (P6), Notification Service (P7), Analytics & Reporting (P8), and Escalation Engine (P9). These processes share three data stores: Users (DS1), Complaints (DS2), and Activity Logs (DS3).

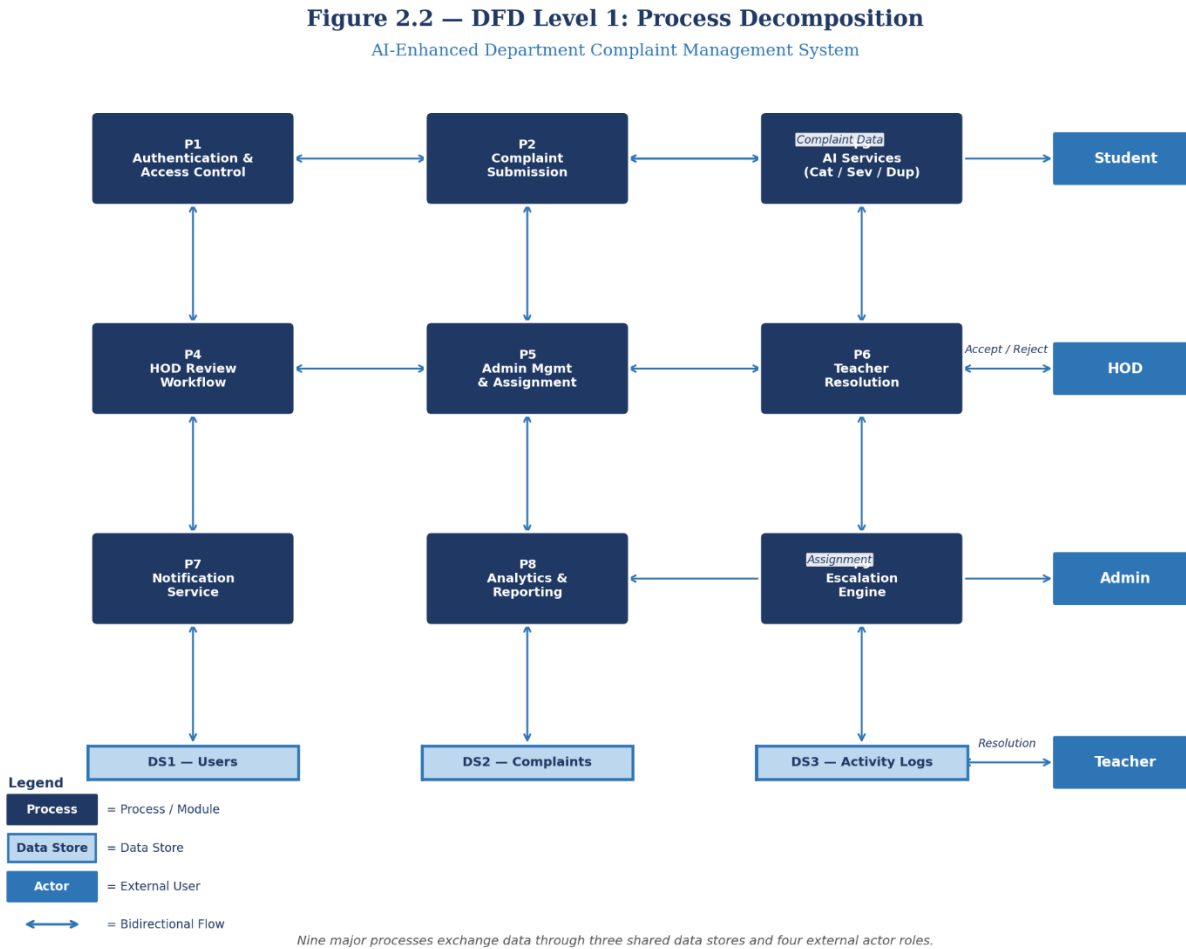


Figure 2.2: DFD Level 1 — Process Decomposition, Complaint Management System

2.3.2 Requirements Engineering

Functional Requirements

- ? User registration, Supabase session login, and role-based dashboard routing for Student, Faculty Member, HOD, DSA, and Supervisor.
- ? Complaint submission with title, description, category/priority support, department routing, optional anonymous flag, and attachment upload.
- ? Automatic routing to HOD or DSA according to category and department rules.
- ? HOD/DSA review with accept/reject actions and faculty assignment.
- ? Faculty work queue with comments and resolved status updates.
- ? DSA/Supervisor finalization, student rating, feedback, and complaint comments.
- ? Notifications, activity logs, analytics, weekly reports, department management, and chatbot assistance.

Non-Functional Requirements

- Security: JWT authentication, role-based permission classes, protected routes.
- Usability: Clear, role-focused dashboards and complaint card interfaces.
- Reliability: Fallback-safe AI modules; graceful degradation when models unavailable.
- Maintainability: Modular backend API structure with clearly separated services.
- Traceability: Activity logging for all key actions.

2.3.3 Deliverables

- Feasibility and requirements specification document.
- DFDs at Level 0 and Level 1.
- UML Use Case and Sequence Diagrams.
- Database design including ERD and normalization to 3NF.
- Implemented full-stack web application with AI services.
- User guide, test evidence, and final thesis report.

2.4 Proposed System Overview

Based on analysis findings, the system is proposed as a centralized, database-driven web platform replacing fragmented manual tools with a single role-governed complaint system. Access is enforced through JWT authentication and role-based permission classes. AI services for category prediction, severity scoring, and duplicate detection are integrated as backend utility modules.

2.5 Assumptions and Constraints

The following assumptions and constraints govern the scope, design, and deployment boundaries of the current version of the AI-Enhanced Department Complaint Management System.

Operational Assumptions

- Users have valid registered accounts with correct role assignments.
- The ML category predictor model file is available; fallback to General if not.
- The sentence-transformer model is available; duplicate blocking disabled gracefully if not.
- Email SMTP configuration is correctly set up for notification delivery.
- System operates as single-department deployment in current version.

Chapter 3

SYSTEM DESIGN

3.1 Introduction to System Design

? To define communication pathways between the Next.js/React frontend, server-side Next.js API routes, Supabase Auth, Supabase PostgreSQL, storage, notification helpers, analytics modules, and chatbot integration.

? To model system interactions using UML Use Case and Sequence Diagrams while keeping the implementation aligned with the current web application.

? To design a normalized relational database for profiles, departments, complaints, attachments, comments, notifications, category routes, and activity logs.

3.2 Proposed System and its Features

The system is organized into three logical layers:

? Layer 1 ? Frontend (Next.js/React): landing, login, registration, role dashboards, complaint form, AI panel, status badges, role badges, analytics, and Botpress chatbot.

? Layer 2 ? Application/API (Next.js API Routes): Supabase session validation, role-based access control, complaint lifecycle actions, staff/department management, notifications, analytics, activity feed, and weekly reports.

? Layer 3 ? Data/Storage (Supabase): PostgreSQL tables, Auth users, Row Level Security policies, and complaint attachment bucket.

3.3 System Design using UML

3.3.1 Use Case Diagrams

Figure 3.1 presents the complete Use Case Diagram for the system. The system boundary encloses all use cases. Five actors are considered in the updated implementation: Student, Faculty Member, HOD, DSA, and Supervisor. Student use cases include Register/Login, Submit Complaint, Upload Attachment, View Status, Add Comment, Rate Resolution, and Use Chatbot Assistant. HOD and DSA use cases include View Routed Complaints, Accept/Reject Complaint, Assign Faculty, Monitor Analytics, and Receive Alerts. Faculty Member use cases include View Assigned Complaints, Add Comments, and Mark Resolved. Supervisor use cases include Full Oversight, Department and Staff Management, Activity Logs, Analytics, Weekly Reports, Assignment Support, and final complaint closure. Notifications are triggered at key workflow events.

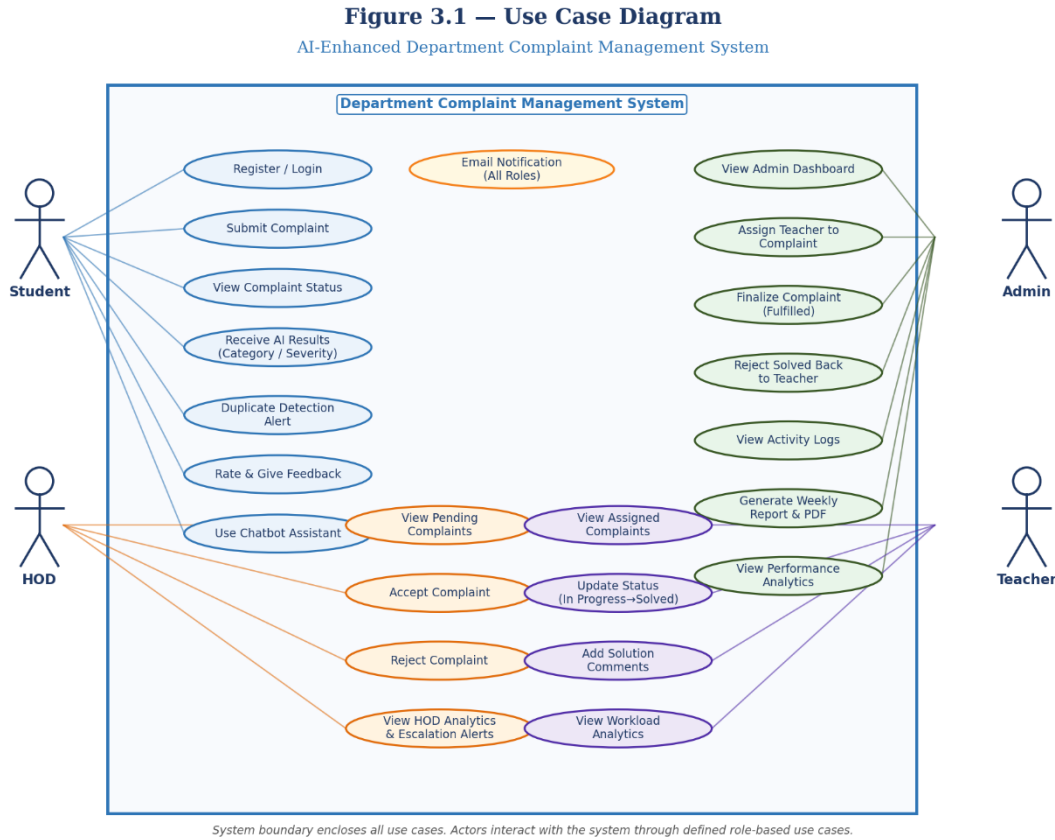


Figure 3.1: Use Case Diagram — AI-Enhanced Department Complaint Management System

3.3.2 Sequence Diagrams

The following five sequence diagrams specify the complete end-to-end behavioral flows for the most critical operational workflows within the system. Solid arrows represent requests and method calls; dashed arrows represent responses and return values.

Figure 3.2 — Sequence 1: Student Login and JWT Authentication

This sequence captures the complete authentication flow when a user logs in. Credentials are submitted to the Django backend, the User table is queried, the password hash is validated, and JWT access and refresh tokens are issued and returned to the frontend for storage and use in subsequent API requests.

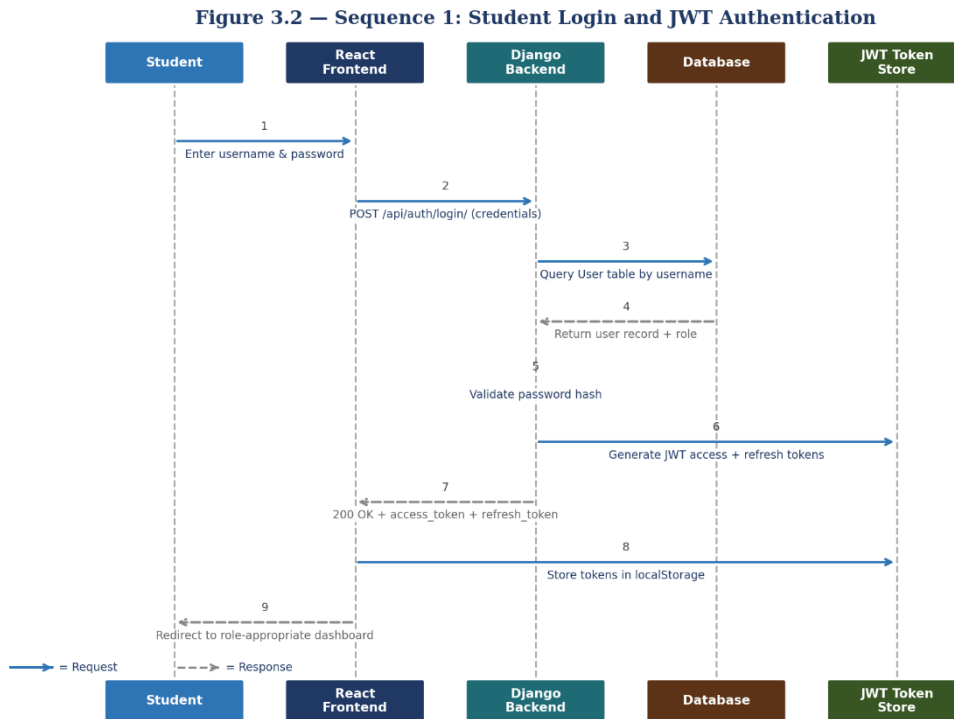


Figure 3.2: Sequence Diagram — Student Login and JWT Token Authentication

Figure 3.3 — Sequence 2: Complaint Submission with AI Processing Pipeline

This sequence models the most complex workflow: AI-assisted complaint submission. Semantic similarity is checked against existing complaints, ML category prediction is applied, severity scoring is computed, the complaint is persisted with Pending status and all AI results, and email notifications are dispatched. A similarity match above the threshold blocks submission and returns a duplicate warning.

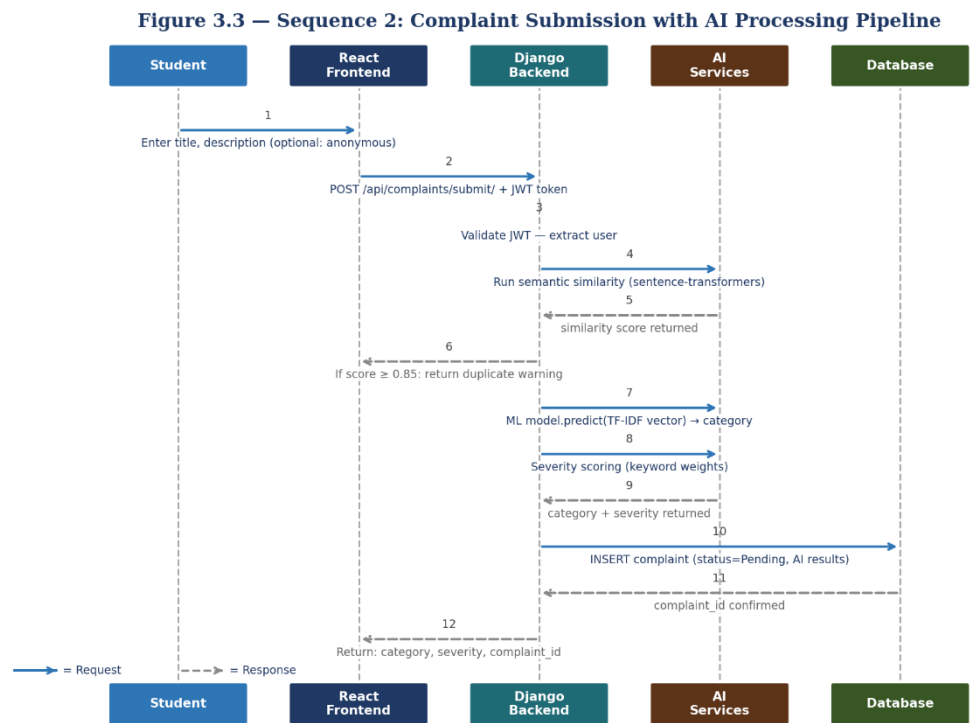


Figure 3.3: Sequence Diagram — Complaint Submission with AI Processing Pipeline

Figure 3.4 — Sequence 3: HOD Review and Admin Teacher Assignment

This sequence models the two-stage complaint routing workflow. HOD reviews and accepts or rejects pending complaints. Accepted complaints move to Ready for Assignment. Admin selects a teacher and submits the assignment. Status advances to In Progress, an ActivityLog entry is created, and the teacher receives an email notification.

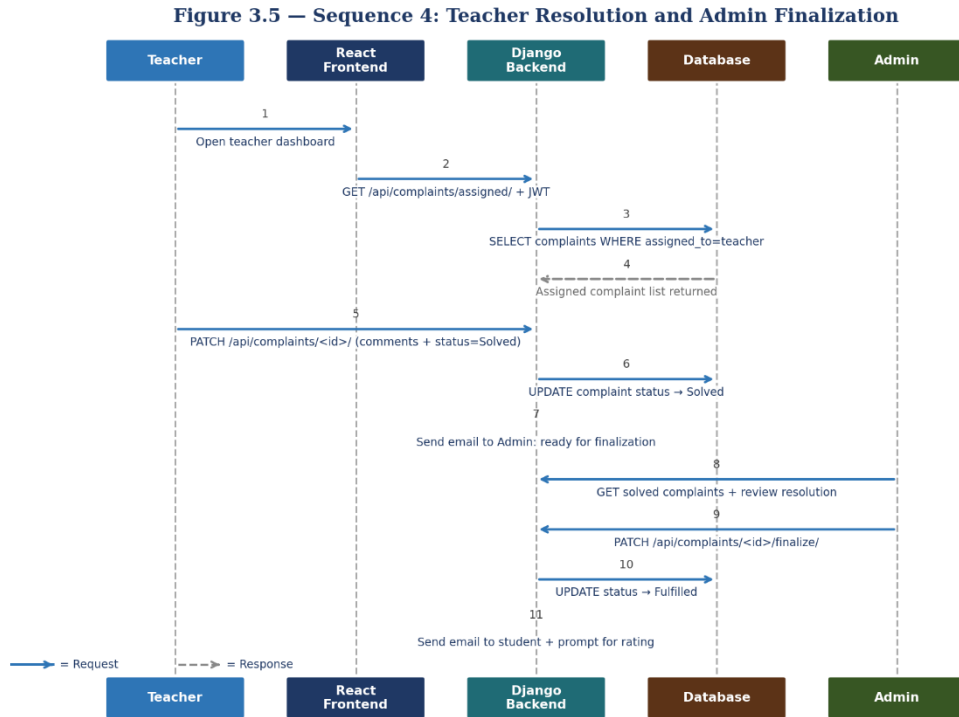


Figure 3.4: Sequence Diagram — HOD Review and Admin Teacher Assignment

Figure 3.5 — Sequence 4: Teacher Resolution and Admin Finalization

The teacher enters solution comments and marks the complaint Solved. Admin reviews the resolution and either finalizes it to Fulfilled (triggering student notification and feedback prompt) or rejects it back to In Progress for further resolution work by the teacher.

Figure 3.5: Sequence Diagram — Teacher Resolution and Admin Finalization

Figure 3.6 — Sequence 5: Scheduled Escalation Engine

A background daemon thread launched via `AppConfig.ready()` runs an infinite loop. On each iteration it queries for unresolved complaints past the escalation time threshold, marks them escalated=True, dispatches HOD alert emails with complaint details, and creates ActivityLog entries. The thread sleeps between iterations until the server process terminates.

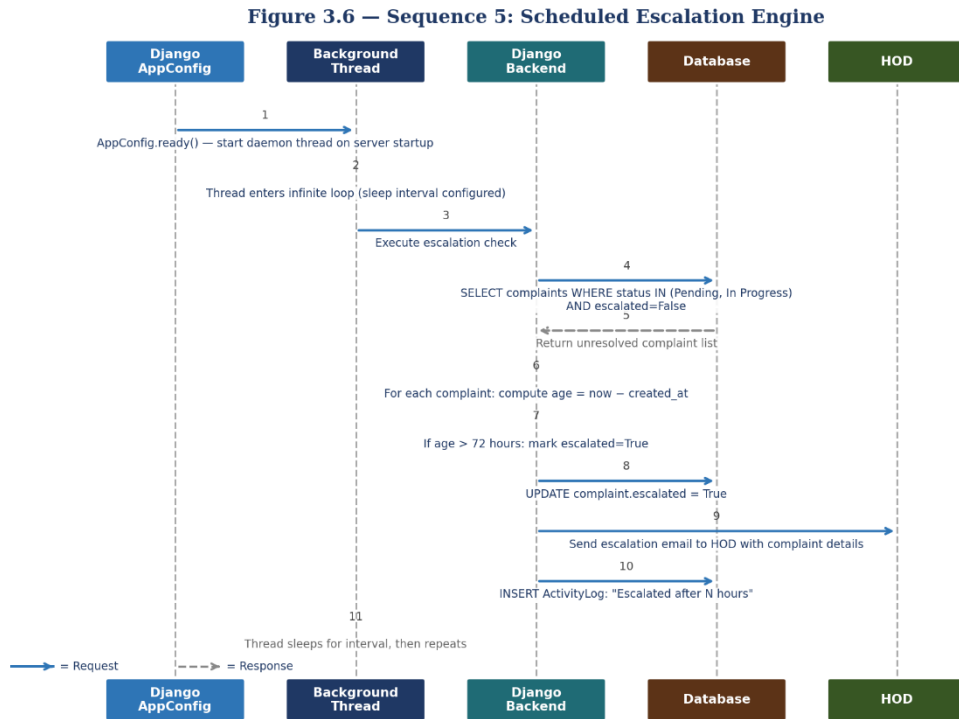


Figure 3.6: Sequence Diagram — Scheduled Escalation Engine (Background Thread)

3.4 Database Design

3.4.1 Entity Relationship Diagram (ERD)

The database design centers on three primary entities: User, Complaint, and ActivityLog. The Complaint entity references the User entity twice — once for the submitting student and once for the assigned teacher. The ActivityLog entity references both User and Complaint. Two additional supporting entities — EmailLog and Ratings — are linked to Complaints to support notification tracking and student feedback.

3.4.2 Relational Model from ERDs

The ERD was systematically translated into a concrete relational database schema by applying standard mapping rules: strong entities map to tables with primary keys; foreign keys enforce inter-table relationships; optional relationships use nullable foreign keys.

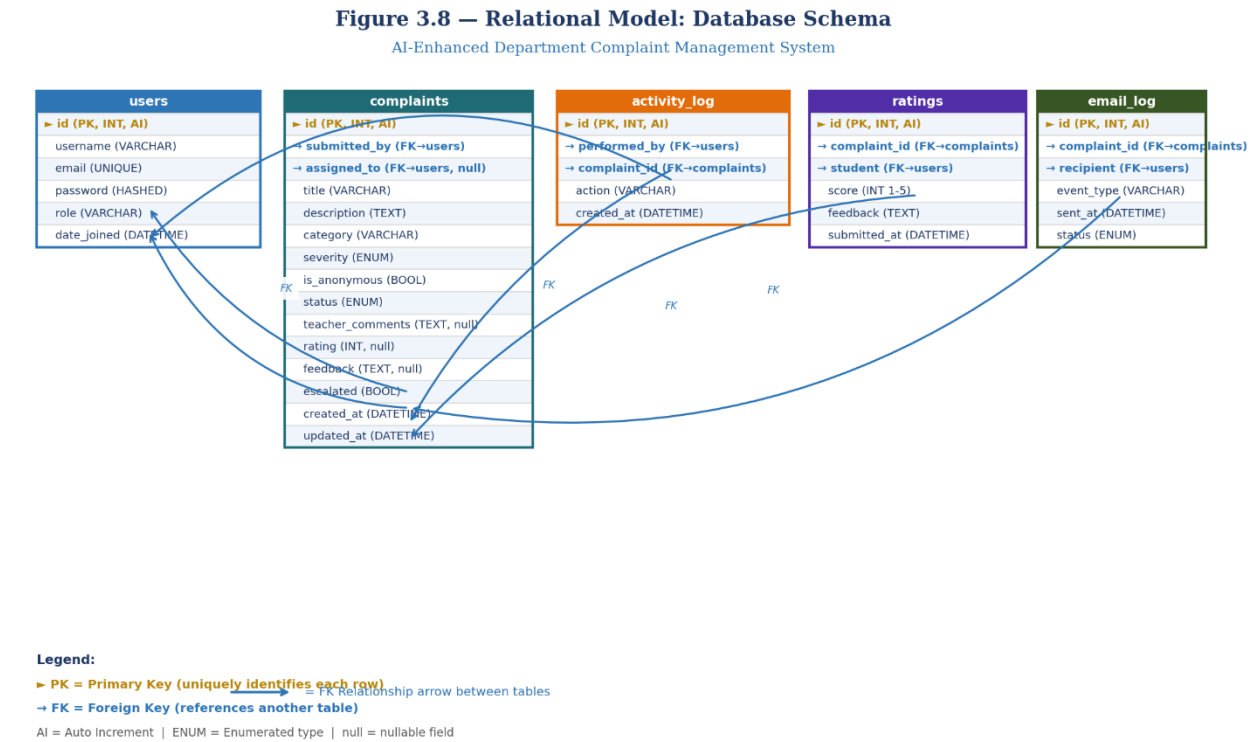


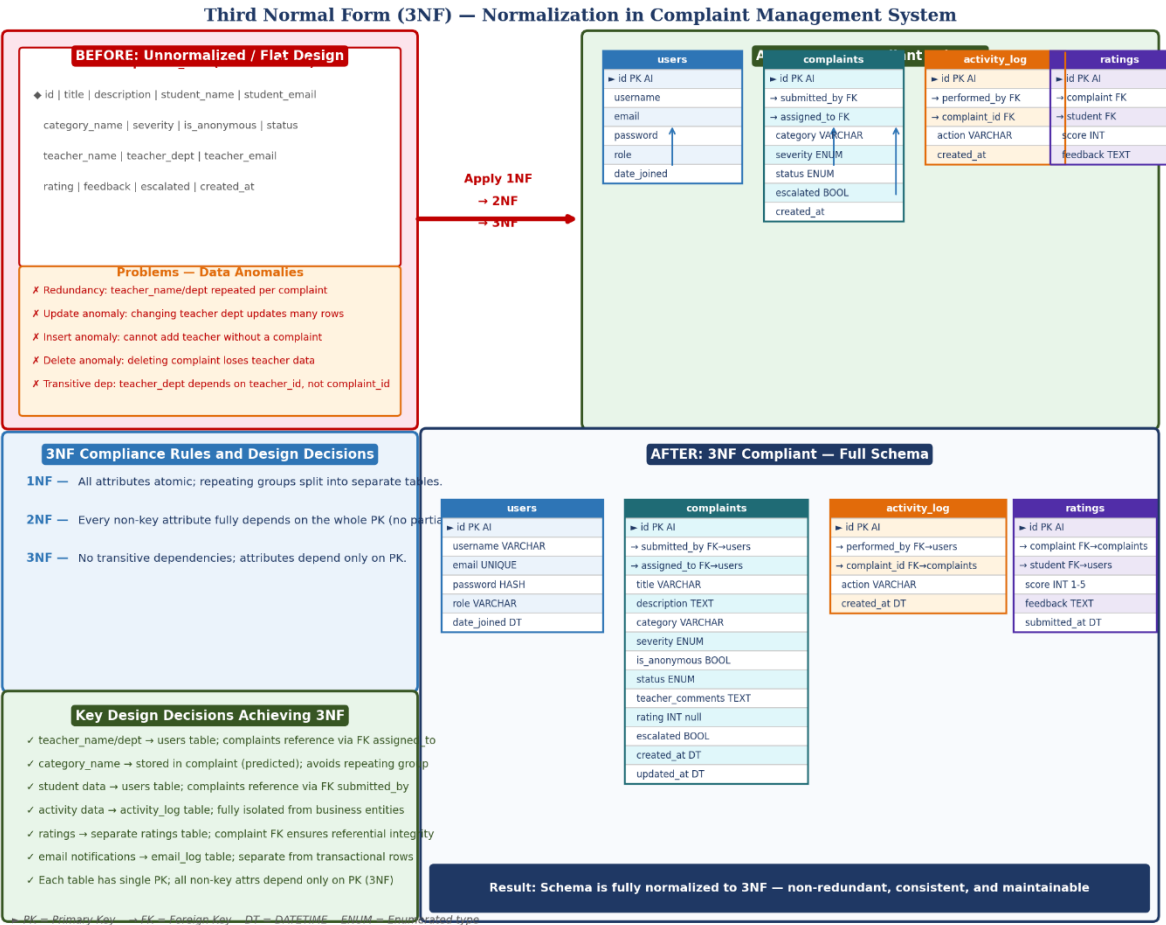
Figure 3.7: Relational Model — Database Schema, Complaint Management System

3.4.3 Normalizing the Relational Model to 3NF

The relational schema was examined and normalized through three normal forms to eliminate data redundancy and update anomalies. The three normalization levels applied are described below with before-and-after diagram illustrations.

First Normal Form (1NF)

1NF requires that all table attributes contain only atomic (single) values and that there are no repeating groups. In the unnormalized complaint data, a single complaint row could contain multiple categories separated by commas, violating atomicity. 1NF is achieved by ensuring each cell contains only one value and splitting multi-valued data into separate rows.



Chapter 4

SYSTEM DEVELOPMENT & IMPLEMENTATION

4.1 Introduction to System Development

The system was implemented following a modular, iterative development approach. Core authentication and complaint workflow modules were implemented first, followed by AI service integration, analytics, reporting, escalation, and chatbot modules.

4.2 Tool and Language Selection

Frontend Framework	Next.js App Router, React, Framer Motion
UI Components	lucide-react, reusable dashboard components
Backend Framework	Next.js Server/API Routes
Authentication	Supabase Auth session tokens
Database	Supabase PostgreSQL
Storage	Supabase Storage complaint-attachments bucket
Analytics/Charts	Recharts and server-side aggregate API routes
Password Security	bcryptjs and validation helpers
Chatbot	Botpress Cloud Webchat
Email/Notifications	Next.js notification/email helper modules
Version Control	Git / GitHub
IDE / Tools	VS Code, Postman

4.3 Hardware Requirements

Minimum CPU	Intel Core i5 or equivalent, 2.0 GHz or above
Minimum RAM	8 GB (16 GB recommended for ML model loading)

Storage	10 GB free space including Python environment and ML models
Network	Internet connection required for Botpress and email service
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 11+

4.4 Software Development and Implementation

4.4.1 Web Portal (Frontend ? Next.js/React)

The frontend is organized around the Next.js App Router with a landing page, login and registration screens, and a unified role-aware dashboard. Reusable components render complaint cards, complaint forms, role badges, status badges, AI suggestions, timelines, analytics, chatbot support, and department management controls.

4.4.2 Backend (Server-Side ? Next.js API Routes)

API routes provide authentication profile lookup, complaints, review, assignment, status updates, finalization, ratings, comments, attachments, notifications, analytics, activity logs, departments, teacher lookup, and weekly reporting. Each protected route validates the Supabase session token and checks the user's role before executing business logic.

4.4.3 Database Technologies

Supabase PostgreSQL stores profiles, departments, complaints, attachments, comments, notifications, category routes, and activity logs. PostgreSQL enums define role, status, category, priority, and routed-role values. Supabase Storage is used for complaint attachments.

4.5 Algorithms of Important Modules

4.5.1 Role-Based Access Control (RBAC) Workflow

15. User logs in through Supabase Auth and receives a session access token.
16. Client requests include Authorization: Bearer token.
17. API route validates the token, loads the profile, and checks allowed roles.
18. Request is rejected with 401/403 when authentication or role scope fails.

19. Complaint queries are scoped by role: Student sees own complaints, Faculty sees assigned work, HOD/DSA see department-routed complaints, and Supervisor sees all.

4.5.2 AI Category Prediction Workflow

- 21. Student enters title, description, category, priority, department, and optional attachment/anonymous flag.
- 22. The system can suggest category and priority before submission.
- 23. Category route is selected from category_routes or defaults: Academic, Facilities, Behavior-related, and Other route to HOD; Administrative routes to DSA.
- 24. Complaint is saved with Submitted status and relevant notifications/activity logs.

4.5.3 Severity Scoring Engine

- 27. Define urgency keyword dictionary with weights: urgent(3), danger(3), harassment(3), deadline(2), broken(2), critical(3), immediate(2).
- 28. Convert complaint description to lowercase.
- 29. Iterate over keywords; add weight to total_score for each match.
- 30. If total_score ≥ 6 : severity = "High".
- 31. If total_score ≥ 3 : severity = "Medium". Else: severity = "Low".

4.5.4 Duplicate Detection (Semantic Similarity)

- 32. Load SentenceTransformer model (all-MiniLM-L6-v2).
- 33. If model unavailable: disable duplicate blocking (graceful fallback).
- 34. Encode new complaint description into a semantic embedding vector.
- 35. Retrieve and encode all existing non-rejected complaint descriptions.
- 36. Compute cosine similarity between new and each existing complaint.
- 37. If max_similarity $\geq$ threshold (0.85): return duplicate_detected = True.

4.5.5 HOD Review and Admin Assignment Workflow

- 38. HOD/DSA opens dashboard and views Submitted complaints within department scope.
- 39. HOD/DSA accepts, rejects, or assigns the complaint to a faculty member.
- 40. Assignment sets the complaint to In Progress and stores assigned_teacher_id.
- 41. Faculty member resolves the complaint with comments; status becomes Resolved.
- 42. DSA or Supervisor finalizes the complaint as Closed; student can rate and add feedback.

4.5.6 Escalation Module

- 44. Analytics and notification APIs identify overdue, rejected, resolved, and pending records so staff can follow up quickly from the dashboard.

- 47. For each: compute age = current_time - created_at.
- 48. If age > threshold (72 hours): mark escalated=True; send HOD email; create ActivityLog entry.

4.5.7 Weekly Reporting Module

- 49. Query Complaint table for current week records.
- 50. Compute total, solved, pending, in-progress counts; average rating; category distribution.
- 51. Return JSON report data to admin dashboard.
- 52. For PDF: ReportLab generates formatted summary with statistics and category table as downloadable attachment.

4.5.8 AI Module Integration Architecture

The three AI modules — category predictor, severity scorer, and similarity detector — are architecturally separated as independent Python service modules within the complaints application directory. This separation provides several important benefits: each module can be independently tested, updated, or replaced without modifying the complaint submission view logic; each module implements its own graceful fallback behavior, ensuring that failure in one module does not propagate to fail the entire complaint submission; and the modular structure makes it straightforward to swap or upgrade individual AI components (for example, replacing the rule-based severity scorer with a trained NLP model) without changing the calling code or the database schema.

The complaint submission view orchestrates the pipeline by calling each module in sequence: (1) `check_duplicate()` from `similarity.py`; (2) `predict_category()` from `complaint_predictor.py`; (3) `score_severity()` from `severity_score.py`. Each function returns a structured result dict with the AI output and a success flag. The view uses the result flags to determine whether to use the AI output or apply the appropriate fallback value before persisting the complaint record.

All three AI modules are designed to be stateless — they receive input text, apply their processing logic, and return output values without maintaining any persistent state between calls. This ensures thread safety when the Django development server runs in multi-threaded mode and prevents request cross-contamination in production WSGI server deployments.

4.5.9 Email Notification Architecture

The email notification service is implemented as a dedicated Python module (`email_service.py`) containing nine distinct notification functions, one for each critical workflow event. Each function accepts the relevant complaint object and recipient user object as parameters, constructs an HTML email template using Python string formatting, and dispatches the email using Django's `send_mail()` function with the configured SMTP backend.

The nine workflow events that trigger email notifications are: (1) Student submission confirmation — sent to the submitting student with the complaint ID, AI-predicted category, and severity results; (2) HOD notification of new pending complaint — sent to the HOD when a new complaint enters Pending status; (3) Student acceptance notification — sent to the student when the HOD accepts their complaint; (4) Student rejection notification — sent to the student when the HOD rejects their complaint; (5) Teacher assignment notification — sent to the assigned teacher with complaint details and a link to their dashboard; (6) Admin notification of solved complaint — sent to Admin when a teacher marks a complaint as Solved; (7) Student fulfillment and feedback notification — sent to the student when Admin finalizes their complaint, prompting satisfaction rating submission; (8) Teacher re-assignment notification — sent to the teacher when Admin rejects a resolved complaint back to In Progress; (9) HOD escalation alert — sent to HOD by the escalation engine when a complaint exceeds the resolution time threshold.

Email functions are called asynchronously where possible using Django's `CONN_MAX_AGE` configuration and thread-safe database connections, preventing email dispatch delays from impacting the API response time experienced by the submitting user

Chapter 5

USER'S GUIDE

5.1 Input Forms

5.1.1 Registration Screen

The Registration Screen is the entry point for all new users of the system. The interface is displayed as a centered card with a clean white background set against a soft blue gradient. At the top, the BZU university logo and the label “BZU Complaint System” identify the platform. Users fill in four required fields — Name, Email Address, Password, and Confirm Password — along with a role selector dropdown that offers Student, HOD, Admin, or Teacher. A checkbox for Terms of Service and Privacy Policy acceptance must be ticked before the account can be created. The primary action button “Create Account →” is styled in solid blue for visibility. A link at the bottom redirects already-registered users to the Login screen.

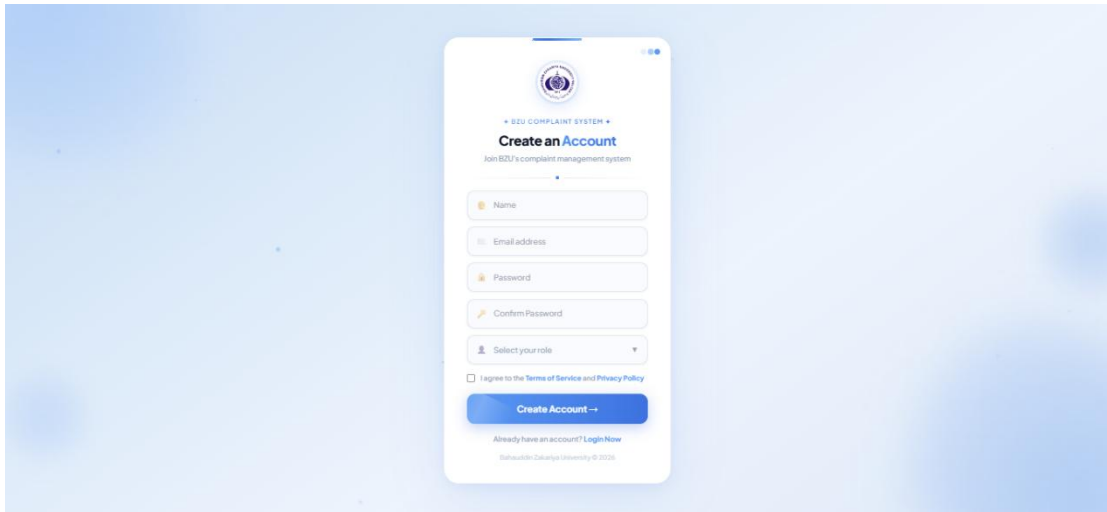


Figure 5.1: Registration Screen

As shown in **Figure 5.1**, the registration form presents a minimal mobile-style card layout. The role selector is critical — selecting the incorrect role will redirect the user to the wrong dashboard after login. All input fields include placeholder text and left-aligned icons for visual clarity. The system validates that both password fields match before submission, and displays inline error messages for any validation failures.

5.1.2 Login Screen (Web)

The Login Screen provides secure access for registered users. The interface follows the same centered card design language as the registration page, with the BZU logo and “Welcome Back” heading greeting returning users. The form requires a Username and Password. A “Forgot

Password?” link is displayed on the right side for account recovery. Below the primary Login button, social sign-in options for Google and Microsoft are offered as alternative authentication methods. Upon successful login, the system reads the user's role from the JWT token payload and automatically redirects to the correct dashboard — Student, HOD, Admin, or Teacher.

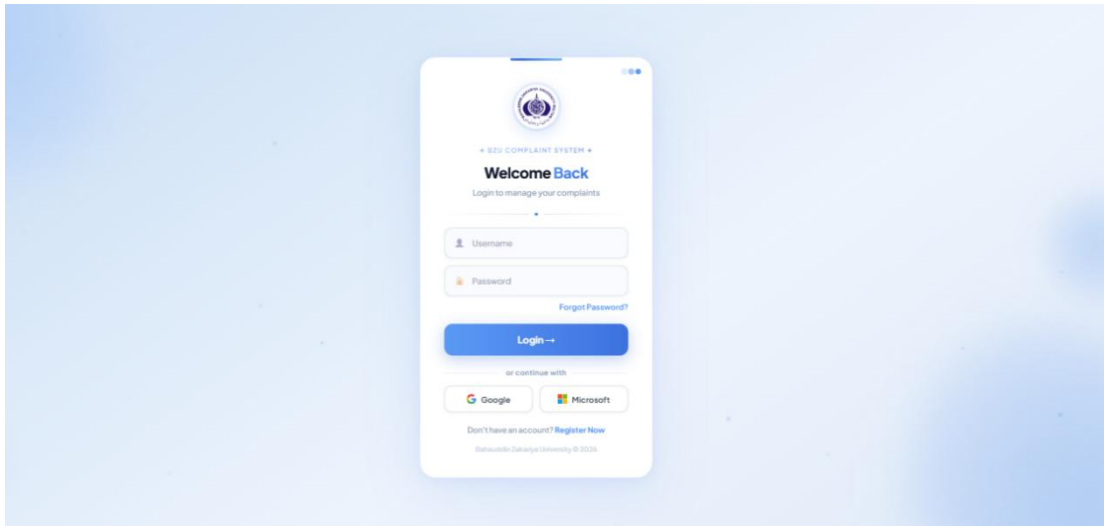


Figure 5.2: Login Screen

As shown in **Figure 5.2**, the login card presents a streamlined two-field layout. The “Login →” button is prominently styled in blue. The Google and Microsoft sign-in options are displayed as bordered secondary buttons with their respective brand icons. Authentication failures trigger an inline error message without refreshing the page, providing immediate feedback to the user.

5.1.3 Complaint Submission Form (Student)

The Complaint Submission Form is embedded directly within the Student Dashboard, making it immediately accessible after login. The form contains a “Complaint Title” field and a larger “Describe your complaint in detail” text area. Below the text area, a “Submit Anonymously” toggle with a person icon allows the student to hide their identity from HOD and Teacher users while Admin retains visibility for accountability. The blue “Submit Complaint →” button triggers the full AI processing pipeline. After submission, a collapsible “AI Analysis Result” panel appears immediately below the form, displaying the predicted Category (e.g., Lab), Severity level (e.g., Low), a Similarity percentage indicating how closely this complaint resembles existing ones, and a complaint ID for future reference.

Good afternoon 🌞

Affifa Fatima,
Student Dashboard
Submit complaints and track their resolution in real-time.

Saturday, April 25
Student Portal

11 Total
3 Solved
5 In Progress
3 Pending

Submit Complaint
Fill in the details below

Complaint Title *

Describe your complaint in detail *

☐ **Submit Anonymously**
Your identity will be hidden

Submit Complaint →

AI Analysis Result

Category: Lab Severity: Low Similarity: 82.01% ID: #39

Figure 5.3: Complaint Form

As shown in **Figure 5.3**, the Student Dashboard presents a greeting header with the student's name and the current date, followed by four statistics cards showing Total, Solved, In-Progress, and Pending complaint counts. The complaint submission form sits below these cards. The AI Analysis Result section visible at the bottom of the figure confirms that category prediction, severity scoring, and duplicate detection all executed successfully on the submitted complaint. This real-time feedback helps students understand how their complaint has been processed before they navigate away from the form.

5.1.4 HOD Review Interface

The HOD Review Interface presents a modal dialog that opens when the HOD clicks the “Review” button on any pending complaint card. The modal displays the full complaint details including the complaint title, submission date, and student identifier. Three AI-generated tags are shown prominently: Severity (e.g., Low), Similarity percentage (e.g., 0.0%), and Category (e.g., Lab). The full complaint description text is displayed in a scrollable content area, allowing the HOD to read the complete complaint before making a decision. Two action buttons at the bottom of the modal — “Reject” in outlined style and “Accept” in solid blue — drive the first-level review workflow.

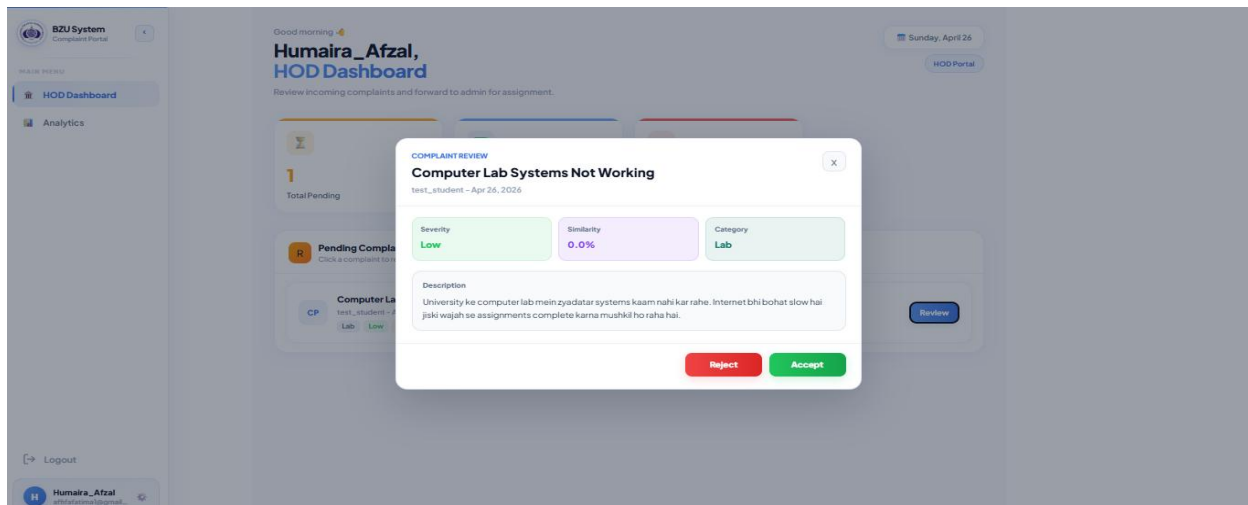


Figure 5.4: HOD Review

As shown in **Figure 5.4**, the HOD Dashboard displays summary counters at the top: Total Pending, Accepted Today, and Rejected Today. Below the counters, the Pending Complaints section lists complaint cards with their title, student name, date, and category/severity tags. The review modal overlays the dashboard and presents the complaint in full detail. The AI-generated severity and similarity scores assist the HOD in quickly gauging the urgency and uniqueness of each complaint before deciding to accept or reject it.

5.1.5 Teacher Assignment Modal (Admin)

The Teacher Assignment Modal appears when the Admin clicks “Assign Teacher” on a ready-for-assignment complaint. The modal presents the complaint title and displays its current tags — category (e.g., Lab), severity (e.g., Low), and complaint ID number. A “Select Teacher” section below shows a list of available teachers with their name and role label. The Admin selects the appropriate teacher by clicking their entry, which highlights the selection. Two buttons at the bottom complete the flow: “Cancel” to dismiss without changes, and “Assign Teacher →” to confirm the assignment. On confirmation, the complaint status transitions to In Progress and an automated email notification is dispatched to the selected teacher.

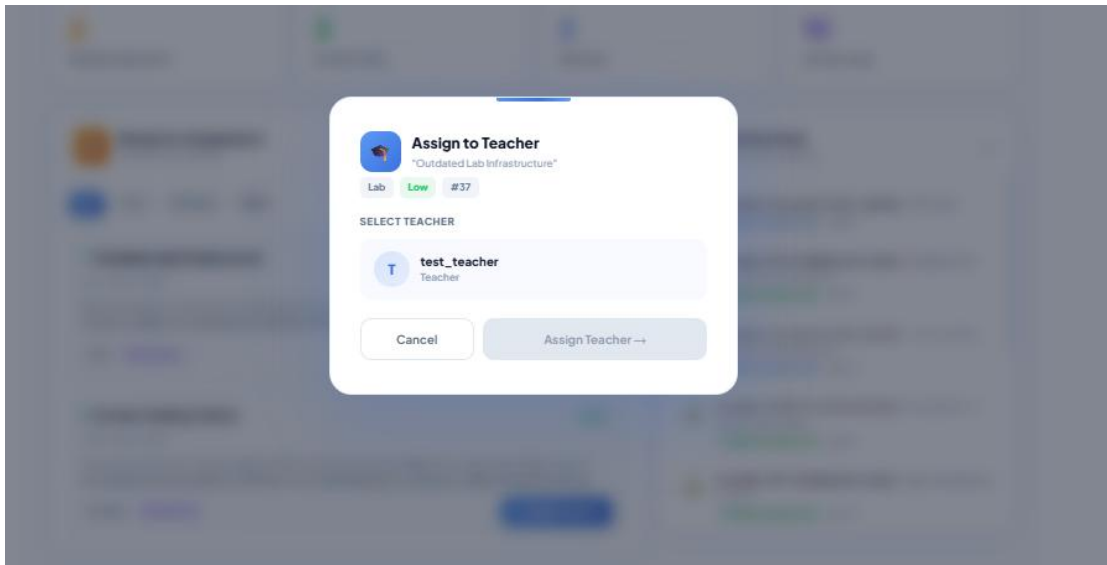


Figure 5.5: Assignment Modal

As shown in **Figure 5.5**, the assignment modal uses a blurred dashboard background to keep focus on the assignment action. The complaint tags (category, severity, and ID) are visible at the top of the modal to help the Admin match the complaint to the most appropriate teacher. The teacher list entry shows both the teacher's avatar initial and their role label, ensuring the Admin can confirm they are assigning to the correct person before submitting.

5.1.6 Teacher Solution / Comment Form

The Teacher Solution Comment Form is a modal dialog that opens when the assigned teacher selects a complaint and chooses to submit their resolution. The modal is titled "Add Solution Comment" and contains a single large text area with the placeholder "Describe the solution in detail", giving the teacher space to document what action was taken to resolve the complaint. Two buttons control the outcome: "Cancel" closes the modal without saving, while "Save Comment" persists the comment and automatically transitions the complaint status from In Progress to Solved. The Admin is notified by email and the complaint appears in the Admin's solved complaints queue for finalization review.

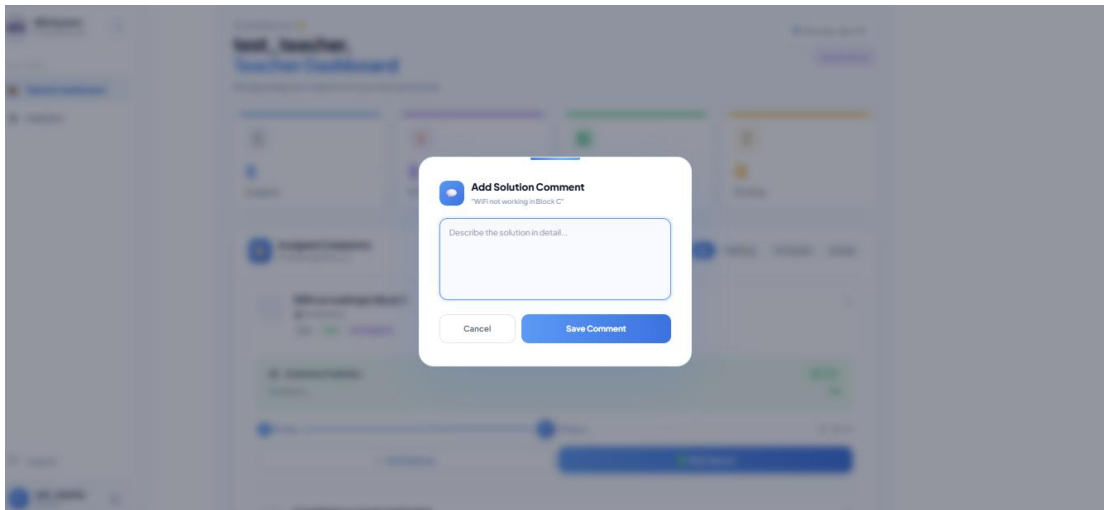


Figure 5.6: Solution Form

As shown in **Figure 5.6**, the solution form modal overlays the Teacher Dashboard with a blurred background. The modal header includes a colored icon identifying the comment type. The text area is generously sized to accommodate detailed resolution notes. The solution comment entered here becomes part of the permanent complaint record and is visible to the Admin during finalization review, ensuring full accountability and transparency in the resolution process.

5.2 Reports and Dashboards

5.2.1 Role-Based Dashboards

Student Dashboard:

The Student Dashboard is the primary interface for student users and is organized into several functional zones. At the top, a personalized greeting displays the student's name (e.g., "Affifa Fatima, Student Dashboard") alongside the current date and a Student Portal badge. Below the greeting, four color-coded statistics cards display real-time complaint counts: Total (blue), Solved (green checkmark), In-Progress (purple lightning), and Pending (orange). The complaint submission form occupies the middle section, followed by a "My Complaints" list with an auto-refresh indicator. A floating blue chatbot button in the bottom-right corner opens the Botpress assistant overlay.

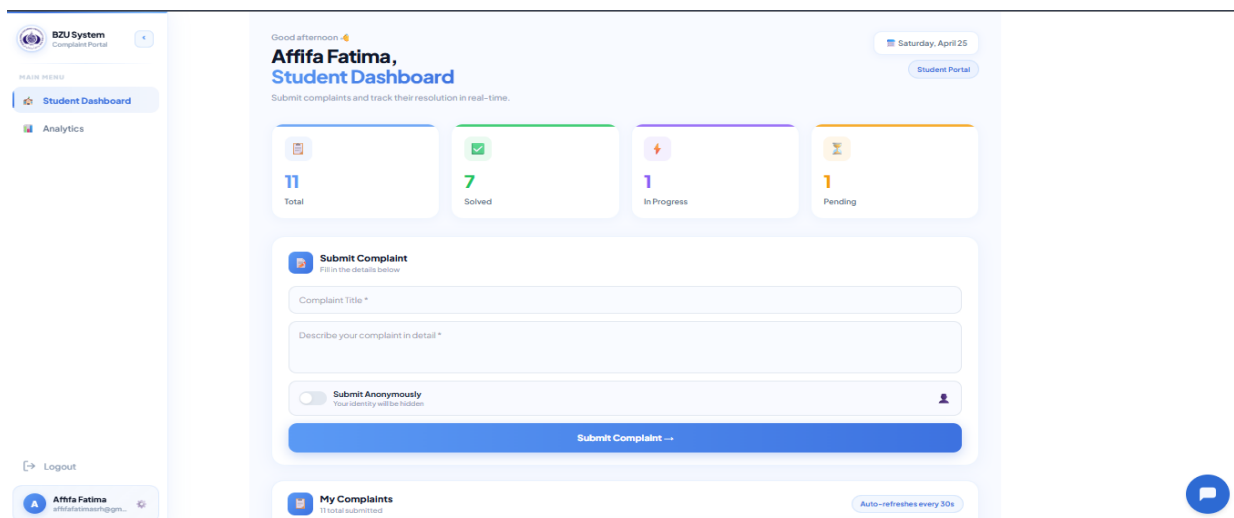


Figure 5.7: Student Dashboard

As shown in **Figure 5.7**, the Student Dashboard presents a clean left navigation sidebar with Student Dashboard and Analytics links. The four statistics cards show live data — in this example: 11 Total, 7 Solved, 1 In-Progress, and 1 Pending. The complaint submission form is immediately visible below the stats cards, enabling the student to submit a new complaint without navigating away from the main dashboard view. The user profile section at the bottom left shows the logged-in student's name and email.

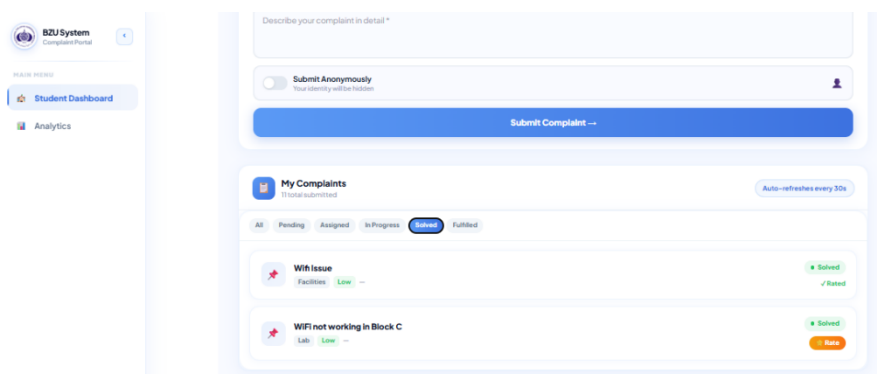


Figure 5.8: Complaint Tracking in Student Dashboard

As shown in **Figure 5.8**, the Complaint Tracking section within the Student Dashboard displays each submitted complaint as a card. Each card shows the complaint title, submission date, category tag, severity tag, and current status badge. A filter bar at the top of the list allows the student to filter by status: All, Pending, In Progress, Solved, or Fulfilled. The “Auto-refreshes every 30s” label indicates that the complaint list updates automatically, so students always see the latest status without manually refreshing the page.

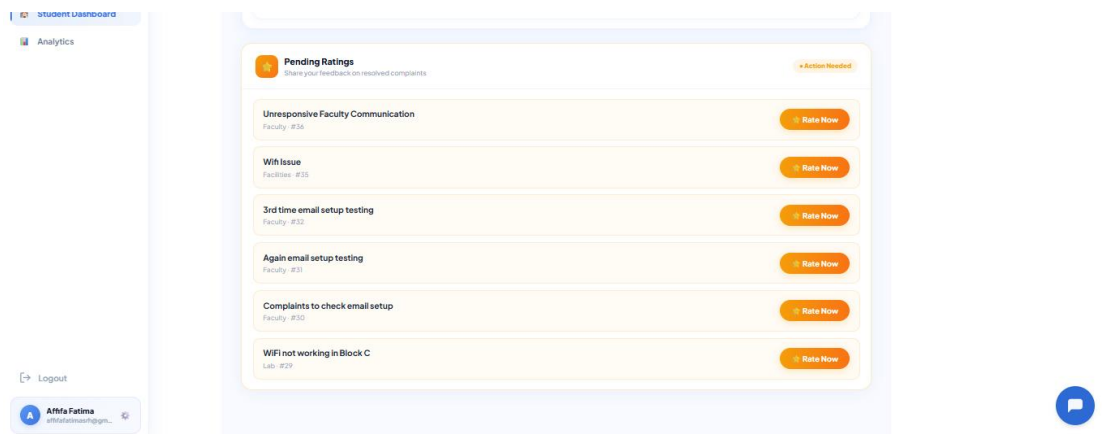


Figure 5.9: Rating Reminder

As shown in **Figure 5.9**, the Rating Reminder section appears below the complaint list when one or more complaints have been finalized but not yet rated by the student. Each unrated complaint is displayed with a prompt to provide feedback. This section ensures students are aware of pending rating tasks and encourages timely feedback submission, which contributes to the Admin's analytics on satisfaction rates.

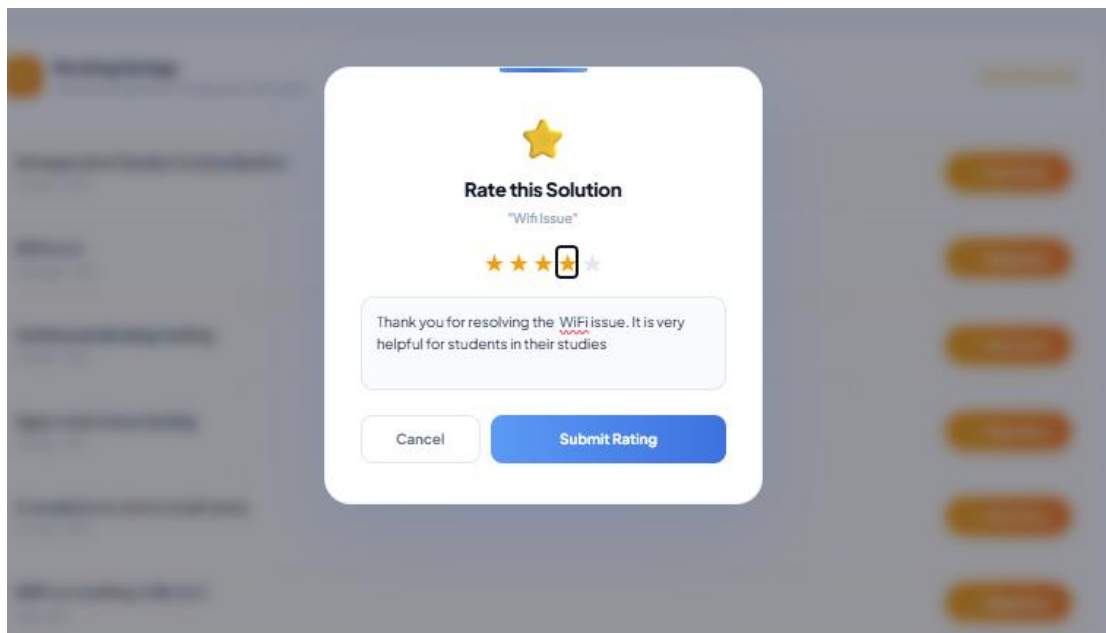


Figure 5.10: Rating Popup

As shown in **Figure 5.10**, the Rating Popup appears when the student clicks on a complaint requiring feedback. The popup presents a star-based rating control alongside a text area for additional written comments. The student can rate the resolution quality on a scale and optionally provide qualitative feedback. Submitted ratings feed directly into the Admin analytics dashboard, contributing to the average satisfaction score shown in the weekly report and role-based analytics panels.

HOD Dashboard:

The HOD Dashboard provides the Head of Department with a focused first-level complaint review interface. The top section displays three summary statistics: Total Pending, Accepted Today, and Rejected Today. Below these counters, the Pending Complaints panel lists all complaints awaiting HOD review. Each complaint card shows the complaint title, submitting student's name, submission date, category tag, severity tag, and a status badge. A “Review” button on each card opens the detailed review modal described in Section 5.1.4.

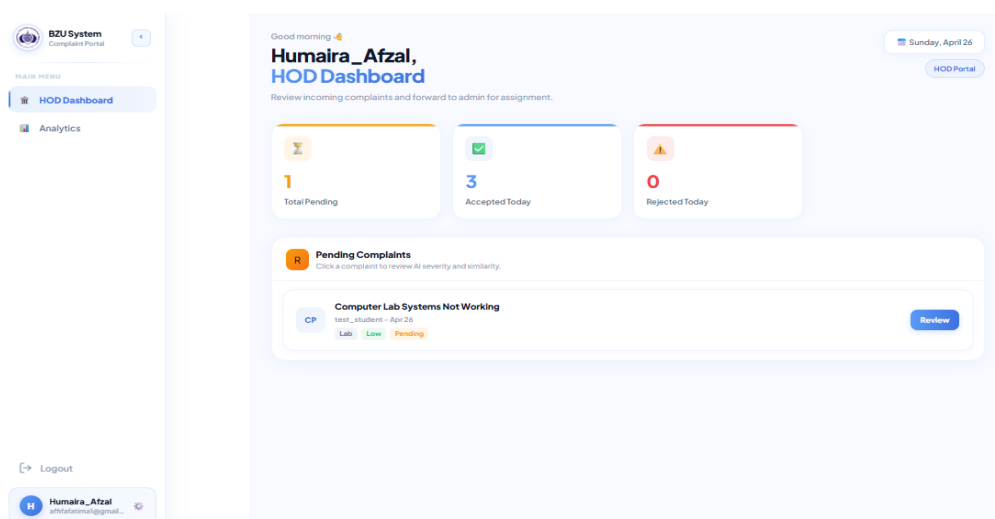


Figure 5.11: HOD Dashboard

As shown in **Figure 5.11**, the HOD Dashboard uses the same sidebar navigation pattern as other role dashboards, with HOD Dashboard and Analytics links. The statistics row at the top shows live counts — in this example: 1 Total Pending, 3 Accepted Today, 0 Rejected Today. The Pending Complaints section below lists complaint cards with all relevant metadata visible at a glance, enabling the HOD to prioritize review based on severity and category without opening the full detail modal.

Admin Dashboard:

The Admin Dashboard is the most feature-rich interface in the system, designed to support complaint assignment, finalization, activity monitoring, and reporting. The dashboard is split into two main columns. The left column displays Ready for Assignment complaints with severity filter tabs (All, Low, Medium, High), showing each complaint's title, description excerpt, category tag, severity badge, and an “Assign Teacher” button. The right column shows the Activity Feed, a real-time chronological log of system actions. Below these two panels, a Solved Complaints section lists all complaints awaiting Admin finalization. A statistics row at the top shows Needs Assignment, Solved Today, Finalized, and Activity Logs counts.

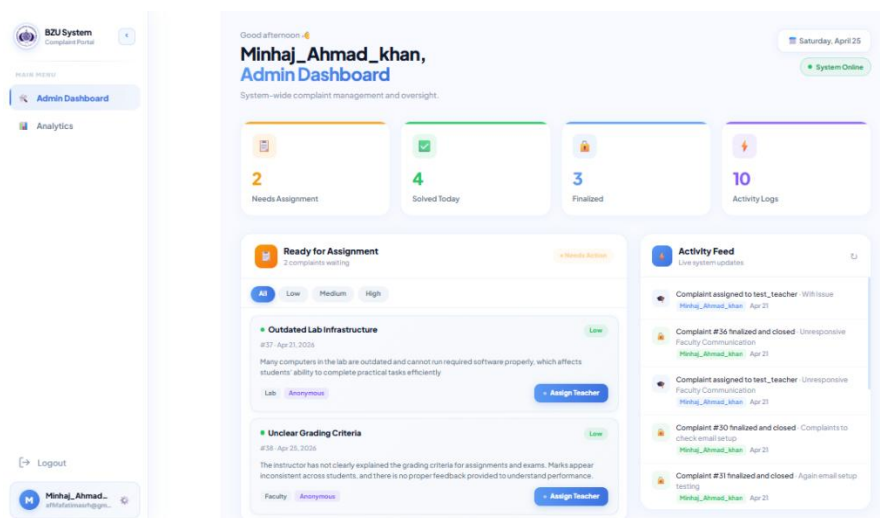


Figure 5.12: Admin Dashboard

As shown in **Figure 5.12**, the Admin Dashboard header shows the admin's name (Minhaj_Ahmad_khan) with an “Admin Online” status badge. The four statistics cards show: 2 Needs Assignment, 4 Solved Today, 3 Finalized, and 10 Activity Logs. The left panel lists two ready-for-assignment complaints — “Outdated Lab Infrastructure” and “Unclear Grading Criteria” — each with their description, tags, and Assign Teacher button. The Activity Feed on the right shows timestamped entries of recent system actions including assignments, finalizations, and rejections, giving the admin real-time awareness of all ongoing complaint workflow activity.

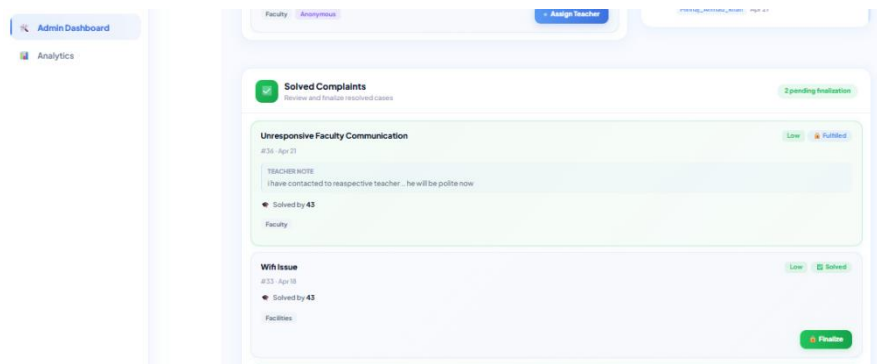


Figure 5.13: Solved Complaints

As shown in **Figure 5.13**, the Solved Complaints section presents complaints that teachers have marked as resolved. Each entry shows the complaint title, the teacher who resolved it, and the solution comment submitted. The Admin can read the resolution details and decide whether the solution is satisfactory. This section feeds directly into the finalization workflow described in Figure 5.14.

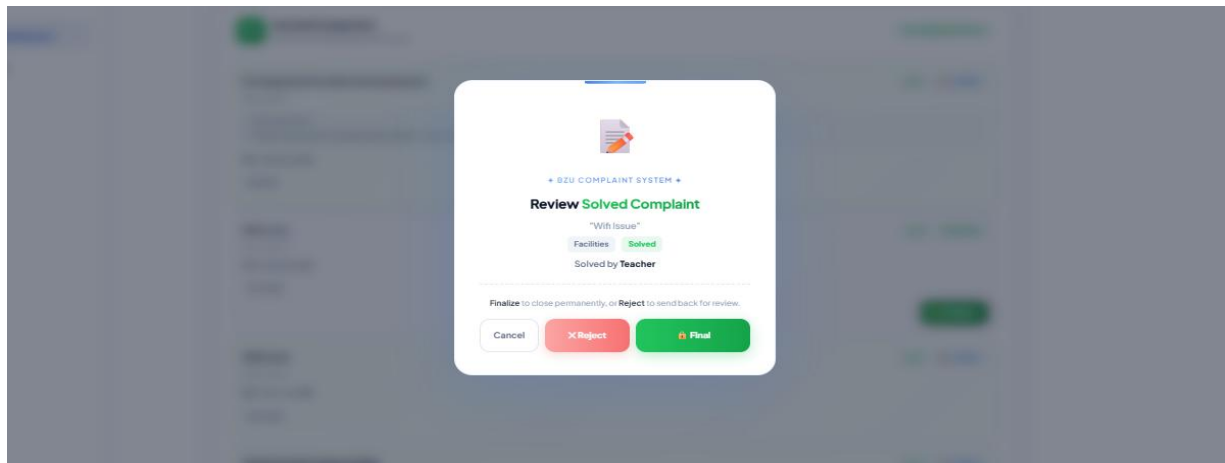


Figure 5.14: Finalization Modal

As shown in **Figure 5.14**, the Finalization Modal overlays the Admin Dashboard when the admin selects a solved complaint for review. The modal displays the complaint title, category tag, and the teacher's solution comment. Two action buttons drive the final decision: "Finalize" marks the complaint as Fulfilled and triggers a student notification with a feedback prompt, while "Reject Back" returns the complaint to In Progress status and notifies the teacher by email to provide an improved resolution.

Teacher Dashboard:

The Teacher Dashboard presents all complaints assigned to the logged-in teacher. A sidebar navigation offers Teacher Dashboard and Analytics links. The main content area displays the teacher's assigned complaint queue with status filter tabs and a solution comment action for each complaint. A workload statistics section provides a quick overview of the teacher's current assignment load and resolution progress.

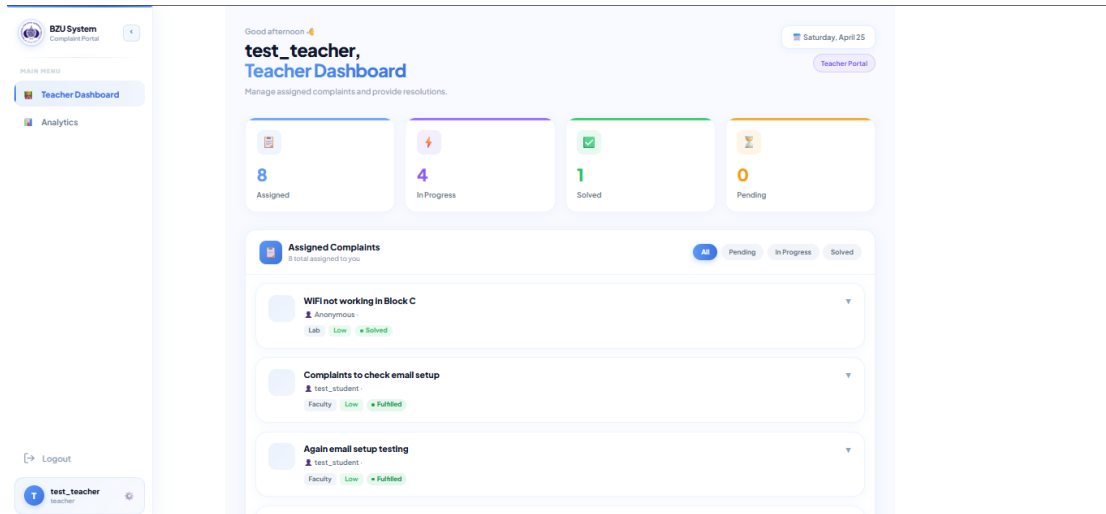


Figure 5.15: Teacher Dashboard

As shown in **Figure 5.15**, the Teacher Dashboard greets the logged-in teacher by name and displays their current workload statistics at the top. The complaint list below shows all actively assigned complaints with their title, date, category, severity, and current status. Action buttons allow the teacher to open the solution comment modal for any in-progress complaint. The dashboard auto-refreshes to reflect any new assignments made by the Admin.

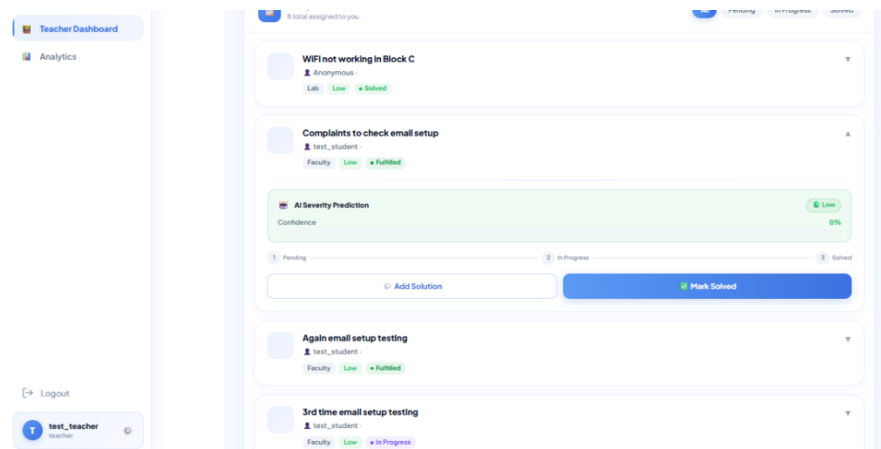
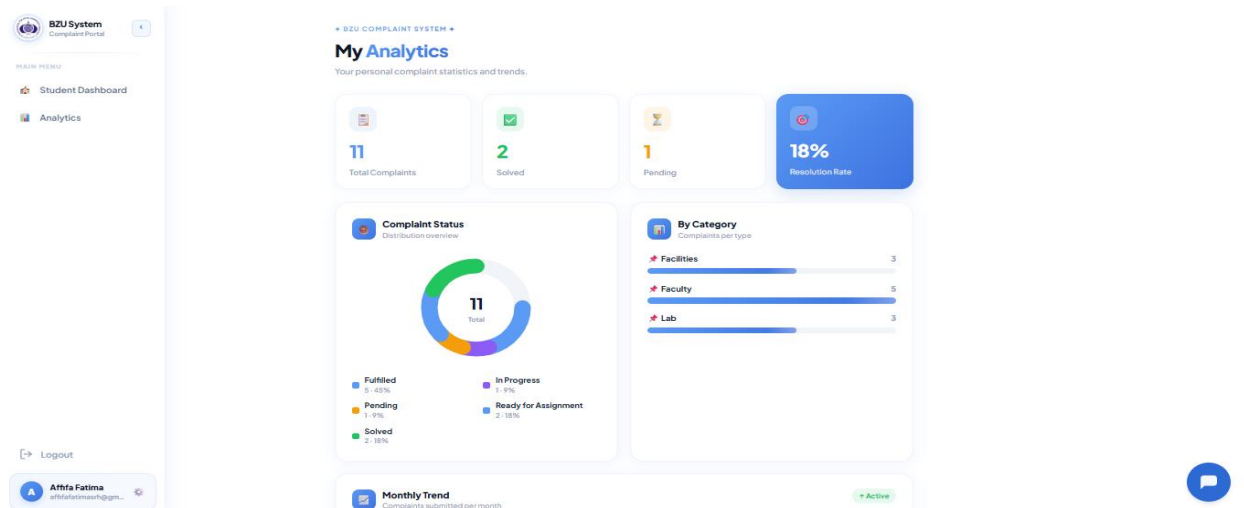


Figure 5.16: Teacher Worklist

As shown in **Figure 5.16**, the Teacher Worklist presents each assigned complaint as a card with full metadata: complaint title, student name, submission date, category tag, severity badge, and status indicator. The teacher can expand any card to read the full complaint description, then open the solution comment modal (Figure 5.6) to document their resolution and mark the complaint as Solved. The worklist supports filtering so teachers can separate In-Progress complaints from those already submitted for Admin review.

5.2.2 Analytic Reports

Student Analytics:

**Figure 5.17: Student Analytics**

As shown in Figure 5.17, the Student Analytics page displays the student's personal complaint data in a structured analytics view titled "My Analytics — Your personal complaint statistics and trends." The top row shows four KPI cards: Total Complaints (11), Solved (2), Pending (1), and a Resolution Rate percentage (18%) displayed in a highlighted blue card. Below the KPI row, two charts are displayed side by side. The left chart is a donut chart showing Complaint Status distribution across Fulfilled, Pending, In Progress, Ready for Assignment, and Solved categories with percentage labels. The right chart is a horizontal bar chart titled "By Category" showing complaint counts broken down by Facilities, Faculty, and Lab categories. A Monthly Trend section at the bottom displays a line or bar chart of complaint submission volume over recent months.

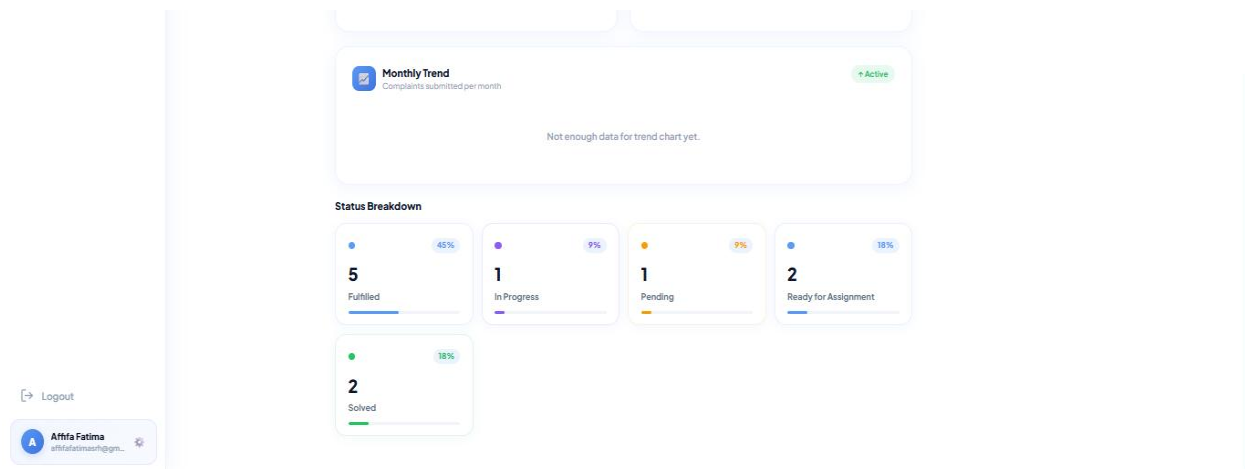


Figure 5.18: Student analytic

As shown in **Figure 5.18**, this view presents an additional analytics panel within the student analytics section, showing the monthly complaint trend chart in detail. The trend chart displays submission activity over the current period, helping the student understand whether their complaint volume has increased or decreased over time and identify patterns in their complaint history.

Teacher Analytics:

The Teacher Analytics section provides workload visibility for teacher users, displaying assigned, pending, solved, and overdue complaint counts.

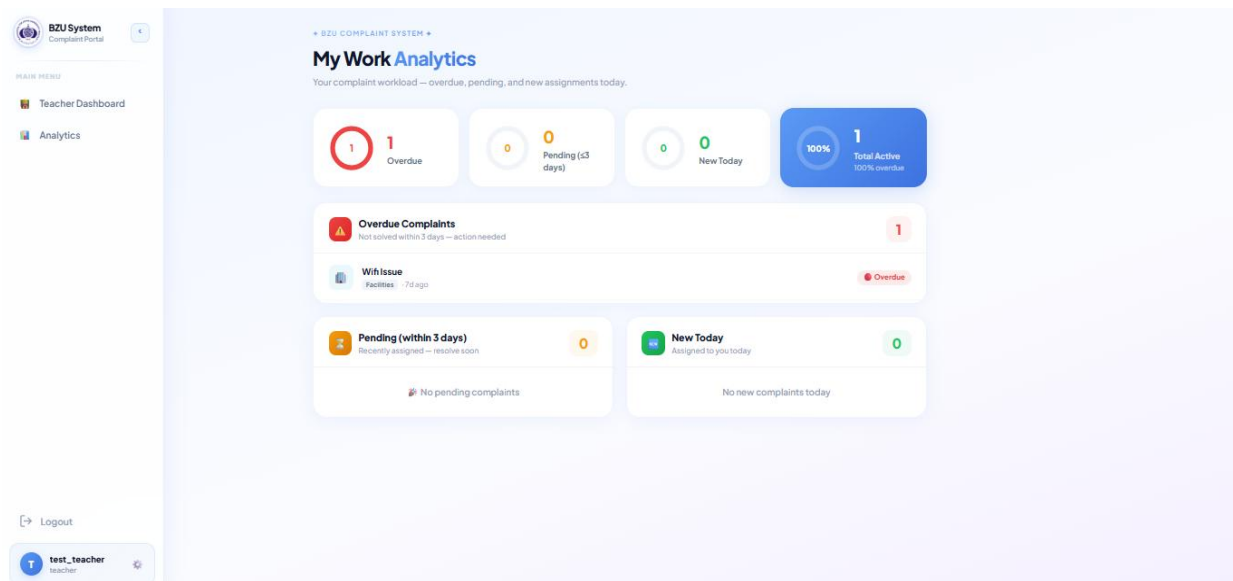


Figure 5.19: Teacher Workload

As shown in **Figure 5.19**, the Teacher Workload analytics panel displays the teacher's active assignment statistics with KPI cards for Assigned, Pending, Solved, and Overdue counts. Bar charts and trend indicators provide the teacher with a visual overview of their complaint resolution performance, enabling them to identify delays and prioritize overdue complaints that may trigger escalation alerts to the HOD.

HOD Analytics:

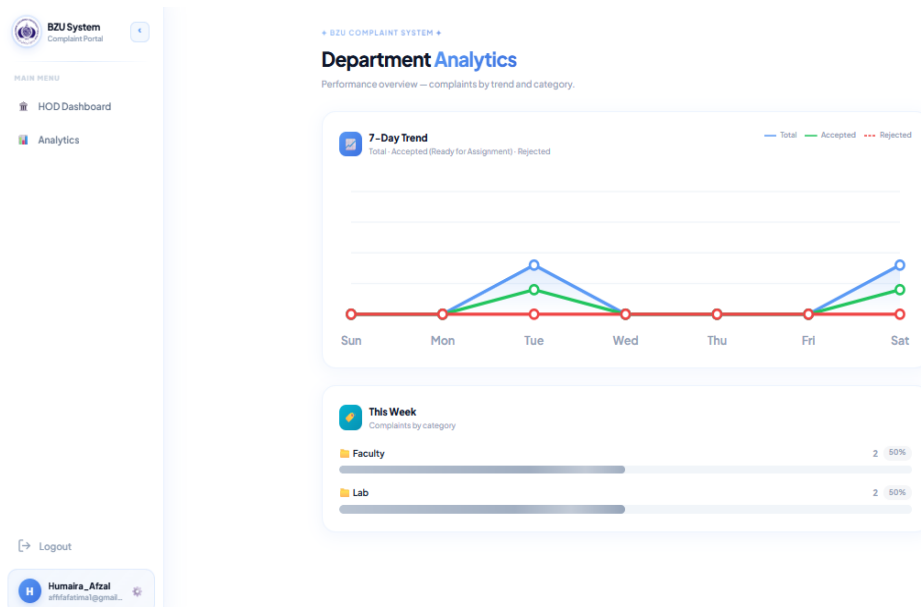


Figure 5.20: HOD Analytics

As shown in **Figure 5.20**, the HOD Analytics page is titled “Department Analytics — Performance overview: complaints by trend and category.” The page displays a “7-Day Trend” line chart with three series: Total (blue), Accepted (green), and Reported (red), plotted across the days of the week from Sunday to Saturday. The chart provides the HOD with a clear visual of complaint volume patterns and acceptance rates across the week. Below the trend chart, a “This Week” section shows a horizontal bar chart of complaints broken down by category (e.g., Faculty at 50%, Lab at 60%), enabling the HOD to identify which departmental areas are generating the most complaints.

Admin Analytics:

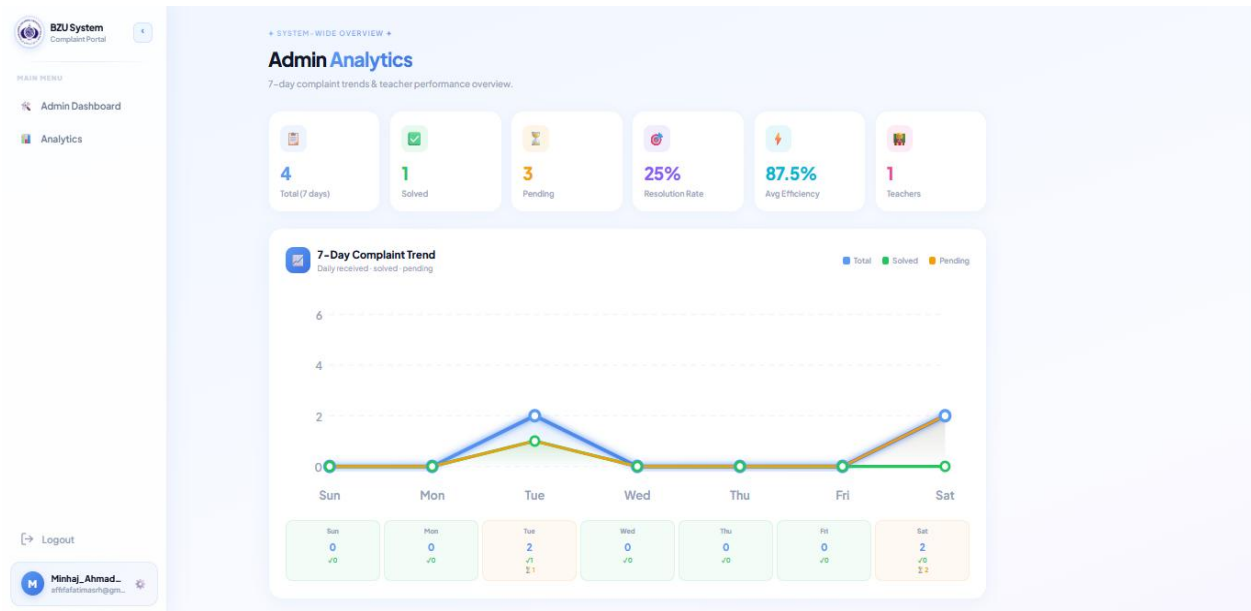


Figure 5.21: Admin Analytics

As shown in **Figure 5.21**, the Admin Analytics dashboard provides a comprehensive system-wide performance overview. The dashboard displays aggregate statistics covering all complaint lifecycle stages, teacher performance metrics, and resolution time data across all active complaints in the system. Multiple chart panels give the Admin a complete picture of how effectively the department is handling student complaints.

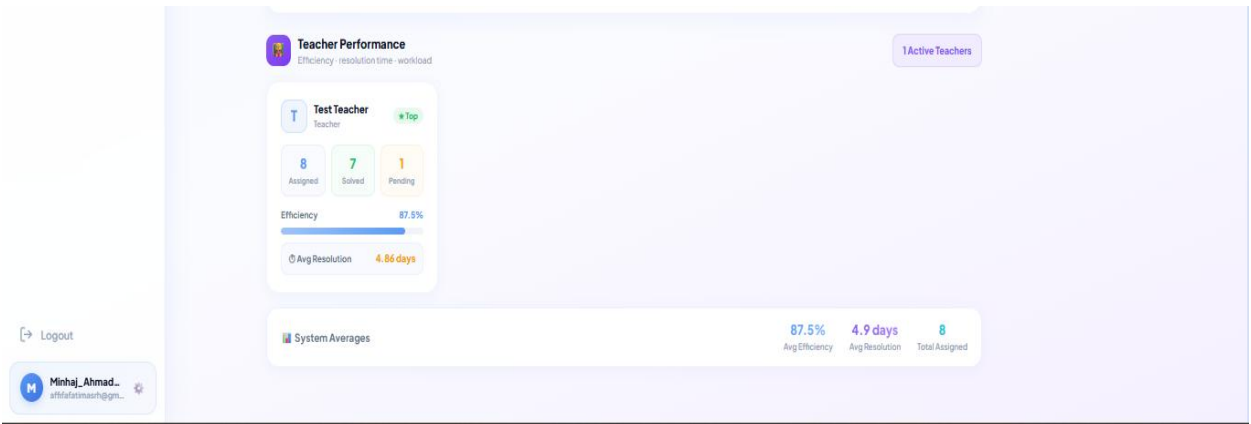


Figure 5.22: Performance Report

As shown in **Figure 5.22**, the Performance Report section within the Admin Analytics dashboard presents a condensed summary of key performance indicators. The report highlights complaint resolution rates, average resolution times, and teacher workload distribution. This data supports data-driven administrative decisions about resource allocation and complaint workflow optimization.

5.2.3 Chatbot and Weekly Reports

The BZU Help Assistant is a Botpress-powered chatbot integrated into the Student Dashboard as a floating overlay panel. When activated, the chatbot panel opens on the right side of the dashboard with a dark header bar showing “BZU Help Assistant” and the subtitle “Ask me anything about the complaint system.” The bot provides structured guidance on how to use the system, covering topics such as how to submit a complaint, what each workflow status means, how the AI categorization works, and how to check complaint progress. A conversation history area displays the bot’s responses in chat bubble format. A text input field at the bottom allows students to type free-form questions. The chatbot is accessible at all times from the student dashboard via the floating blue chat button in the bottom-right corner.

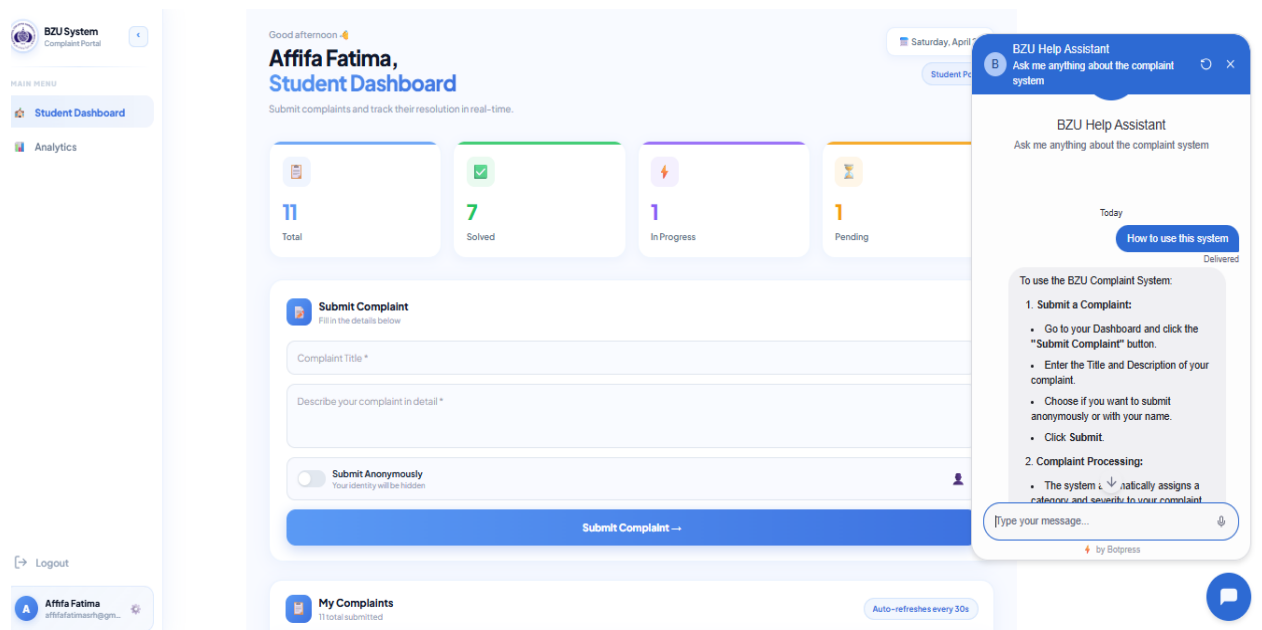


Figure 5.23: Chatbot Assistant

As shown in **Figure 5.23**, the chatbot panel is visible on the right side of the Student Dashboard. The conversation displayed shows the bot providing a step-by-step guide on how to use the BZU Complaint System, including instructions for submitting a complaint, understanding AI processing results, and tracking status. The student can ask follow-up questions in the message input field at the bottom of the panel. The floating chat button remains visible even when the panel is closed, ensuring the student always has quick access to assistance.

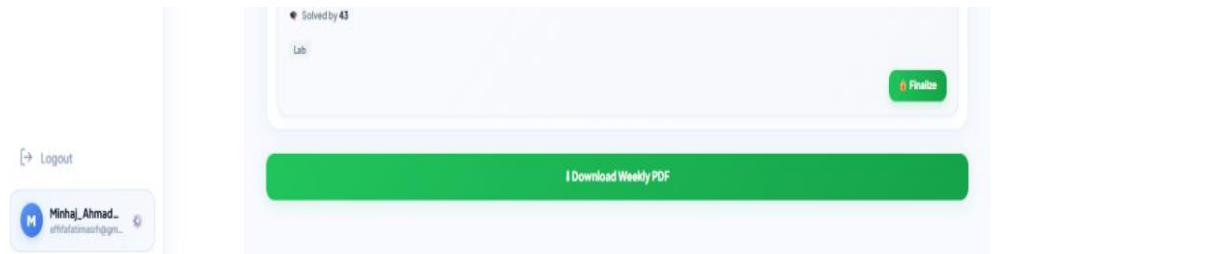


Figure 5.24: Weekly Report

As shown in **Figure 5.24**, the Weekly Report interface is accessible from the Admin Dashboard. The report displays a summary of complaint activity for the current week including total complaints, solved cases, pending cases, and in-progress counts. A prominent green “Download Weekly PDF” button at the bottom allows the Admin to generate and download a formatted PDF report created using the ReportLab library. The PDF report includes the statistical summary table and category distribution data, formatted for departmental record-keeping and review meetings.

5.2.4 Activity Feed and History

The Activity Feed is a real-time chronological log of all complaint workflow actions within the system, accessible from the Admin Dashboard. Each activity entry records: the action type (e.g., assigned, finalized and closed, rejected and sent back), the complaint title or ID involved, the name of the user who performed the action, and a relative timestamp such as “19 hours ago” or a specific date. The feed updates automatically as new actions are performed, giving the Admin continuous visibility into all ongoing complaint handling activity without needing to open individual complaint records. The Activity Feed entries correspond directly to the ActivityLog database records created by the backend at each key workflow transition, providing a full audit trail of the complaint lifecycle.

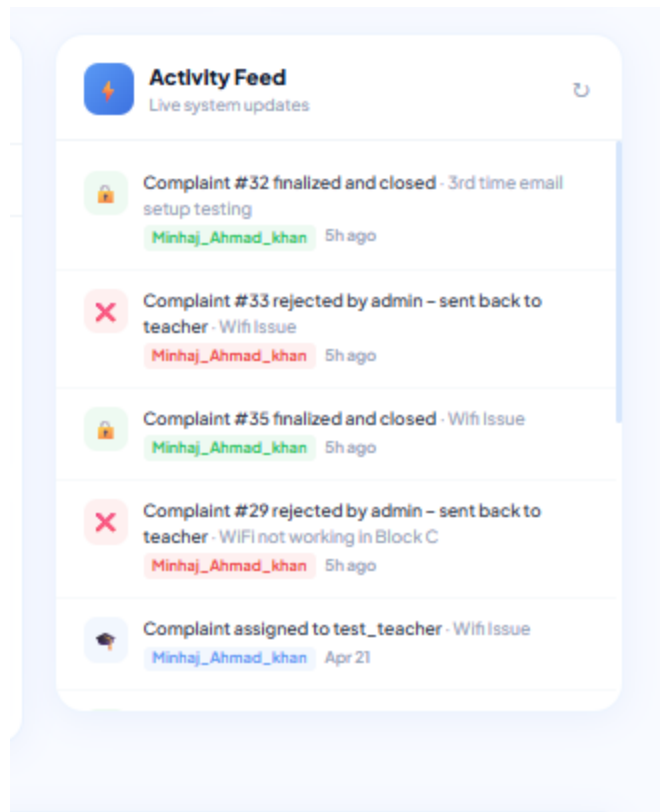


Figure 5.25: Activity Feed

As shown in **Figure 5.25**, the Activity Feed panel within the Admin Dashboard lists recent system actions in chronological order. Visible entries include “Complaint assigned to test_teacher,” “Complaint #3 finalized and closed,” and “Complaint rejected and sent back,” each attributed to a named admin user with a relative timestamp. The feed provides the Admin with immediate awareness of all assignment and finalization actions, supporting accountability and making it easy to identify the most recently processed complaints at a glance.

5.2.5 AI Processing Results Report

The AI Processing Results Report is embedded within the Admin Analytics dashboard and provides quantitative insight into the performance of the three AI pipeline modules: category prediction, severity scoring, and duplicate detection. The report shows: the total number of complaints processed through the AI pipeline; a category prediction distribution table breaking down how many complaints were assigned to each category (e.g., Lab, Faculty, Facilities, Admin, IT) by the scikit-learn ML model; a severity level distribution showing the count and percentage of Low, Medium, and High severity assignments produced by the keyword-based scoring engine;

and duplicate detection statistics covering total submission attempts in the current month, total blocked duplicates, and the calculated block rate percentage.

This data allows administrators to evaluate whether the AI pipeline is performing as expected. A high concentration of complaints in a single category (e.g., Lab) may indicate a recurring departmental issue requiring targeted intervention. An unusually high or low duplicate block rate may signal that the similarity detection threshold needs recalibration. determine whethe

Chapter 6

CONCLUSION

6.1 Project Summary

6.1.1 Summary of Work Completed

- ? Full-stack web application with Next.js/React frontend and Next.js API backend.
- ? Supabase Auth based authentication with role-based routing and permission checks.
- ? Supabase PostgreSQL schema for profiles, departments, complaints, attachments, comments, notifications, category routes, and activity logs.
- ? Structured complaint lifecycle: Submitted, In Progress, Resolved, Closed, Rejected, and Escalated.
- ? Dashboards for Student, Faculty Member, HOD, DSA, and Supervisor.
- ? Notifications, analytics, weekly reports, chatbot support, and attachment handling.

6.2 Project Outcomes Summary

- Clear and traceable multi-role complaint lifecycle from submission to closure.
- Practical AI support at point of complaint submission, reducing manual categorization.
- Reduced redundancy through semantic duplicate detection.
- More informed routing through AI-generated category and severity results.
- Increased transparency through complaint status tracking and activity logging.
- Better administrative insight through analytics and weekly reports.

6.3 Key Achievements

- Successfully integrated three AI techniques (ML classification, rule-based scoring, semantic similarity) into a single cohesive pipeline.
- Designed and implemented modular backend architecture with clean separation of services.
- Implemented fallback-safe AI modules ensuring system reliability when models unavailable.
- Built complete RBAC with server-side enforcement via Django permission classes.

6.4 Limitations

- ? Attachment support is implemented through Supabase Storage with allowed file types.

- Limited automated test suite; validation is primarily scenario-based.
- SQLite not optimized for high-scale production concurrent deployment.
- Single-department deployment; multi-department support requires extension.

6.5 Future Work and Recommendations

Based on the experience gained during the design and implementation of this project and the limitations identified in Section 6.4, the following future enhancement directions are proposed for subsequent versions of the system:

- Add document and image attachment upload support for complaints.
- Implement real-time notifications through WebSockets or push services.
- Replace SQLite with PostgreSQL for production deployment.
- Improve severity scoring using a data-driven NLP model.
- Develop a mobile application for student complaint submission.
- Add multilingual support including Urdu and English.
- Deploy using cloud infrastructure with CI/CD pipelines.

6.6 Personal Reflection

This Final Year Project represented one of the most comprehensive and intellectually demanding undertakings of the undergraduate computer science program. The project required simultaneous engagement with multiple technical domains: backend API design and REST principles, frontend component architecture and state management, database modeling and normalization theory, machine learning model training and inference integration, natural language processing with semantic embeddings, automated email notification service design, background task scheduling, PDF generation, chatbot integration, and full-cycle software engineering documentation. Managing these domains within a single cohesive project required careful architectural thinking and disciplined module separation from the earliest design decisions.

This project provided comprehensive hands-on experience in full-stack web development, AI service integration, software architecture design, and technical documentation. The integration of

machine learning, semantic NLP, and rule-based AI within a real-world workflow management system demonstrated how multiple AI techniques can complement each other. The project reinforced the importance of modular design, graceful error handling, and role-based security in maintainable software systems.

References

1. Next.js Documentation. <https://nextjs.org/docs>
2. React Documentation. <https://react.dev/>
3. Supabase Documentation. <https://supabase.com/docs>
4. Supabase Auth Documentation. <https://supabase.com/docs/guides/auth>
5. Supabase Storage Documentation. <https://supabase.com/docs/guides/storage>
6. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
7. Recharts Documentation. <https://recharts.org/>
8. Framer Motion Documentation. <https://www.framer.com/motion/>
9. Botpress Cloud Documentation. <https://botpress.com/docs>
10. Pressman, R.S. (2014). Software Engineering: A Practitioner's Approach (8th ed.).
11. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

Appendix A: Technology Stack Summary

Frontend	Next.js App Router, React, Framer Motion
UI/Icons	lucide-react, custom reusable components
Backend	Next.js API Routes
Authentication	Supabase Auth session tokens
Database	Supabase PostgreSQL
Storage	Supabase Storage complaint-attachments bucket
-Analytics	Recharts and API aggregate endpoints
Password Security	bcryptjs and password validation helpers
Chatbot	Botpress Cloud Webchat
Email/Notifications	Next.js notification/email helper modules
Version Control	Git / GitHub

Appendix B: Key Implemented Backend Files

? src/app/api/complaints/route.js ? Complaint create/list endpoint.
? src/app/api/complaints/[id]/review/route.js ? HOD/DSA review actions.
? src/app/api/complaints/[id]/assign/route.js ? Faculty assignment workflow.
? src/app/api/complaints/[id]/status/route.js ? Faculty status update workflow.
? src/app/api/complaints/[id]/finalize/route.js ? Complaint closure workflow.
? src/app/api/complaints/[id]/rate/route.js ? Student rating and feedback.
? src/app/api/complaints/analytics/route.js ? Dashboard analytics.
? src/app/api/complaints/weekly-report/route.js ? Weekly report data.
? src/app/api/departments/route.js ? Department management.
? src/lib/workflow.js ? Role scoping and category route defaults.
? src/lib/auth.js ? Supabase session validation and role checks.
? supabase/schema.sql ? PostgreSQL schema, enums, indexes, RLS, and storage bucket.